

SCRMS 39 Feature Data Flow Diagrams

Diagram-first viva guide for understanding how each feature works

Source PDF referenced: D:\Phase 13\SCRMS__Explanation_Detailed_With_Code.pdf

Workspace source: D:\S\scrms

How to read these DFD pages

External Actor

Who starts the feature or receives the result.

Frontend Process

React page/component/hook where the user action happens.

API / Route

HTTP endpoint or Socket.io path used to cross into backend logic.

Backend Logic

Controller, middleware, service, scheduler, or socket handler.

Database / Data Store

MongoDB model, browser cache, or persistent storage touched.

External Service / Realtime

Razorpay, Cloudinary, CloudConvert, email, push, cron, or Socket.io.

Core pattern

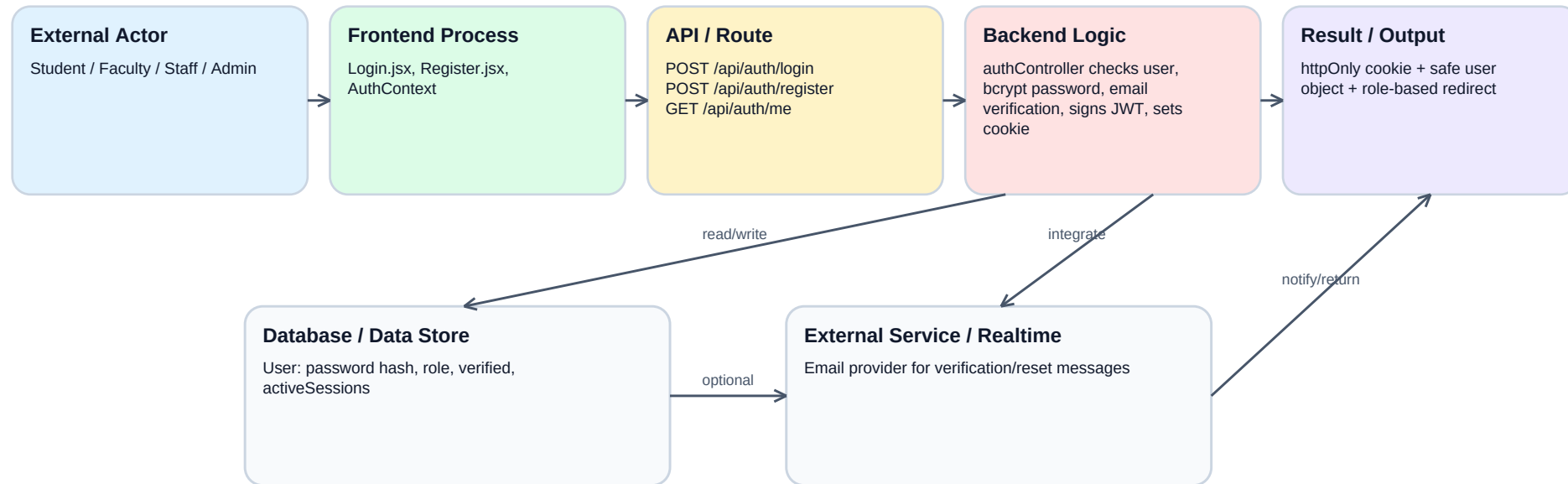
User action -> React screen -> Axios/API call -> Express route -> controller/service -> Mongoose model -> MongoDB/external service -> JSON/Socket/push response -> UI updates.

Viva tip

When an examiner asks any feature, answer using the same sequence: who starts it, which frontend screen sends it, which backend route/controller handles it, which model stores it, and what output the user sees.

Feature 1 - Authentication (JWT + Bcrypt)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



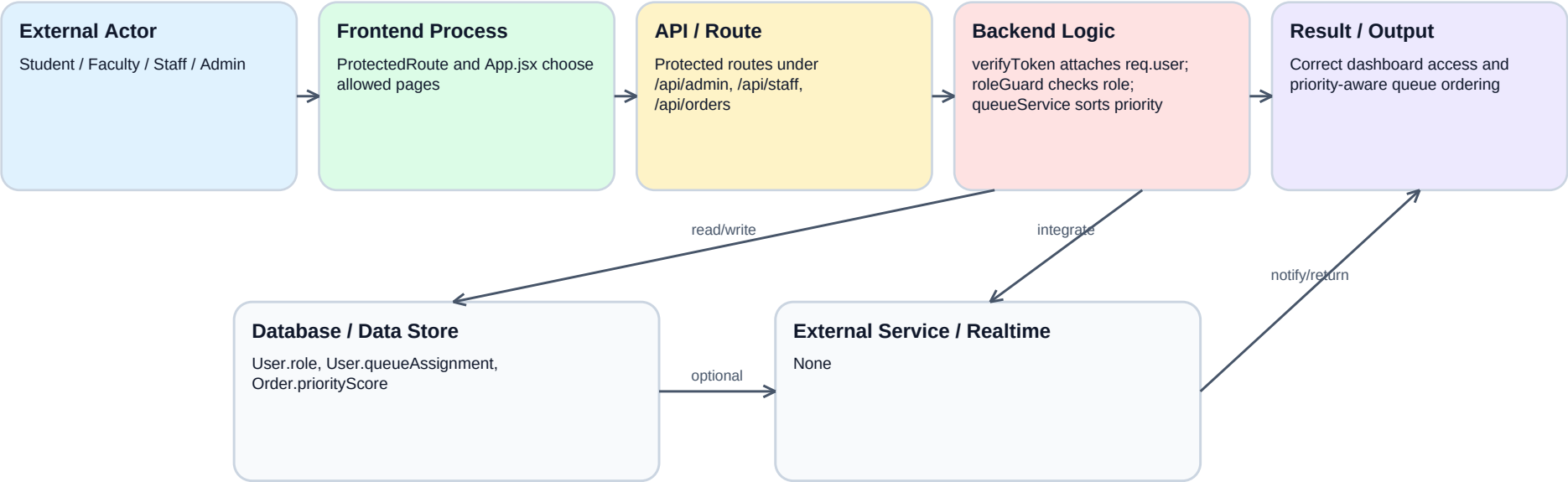
How to explain this feature

I start from Student / Faculty / Staff / Admin. The UI part is Login.jsx, Register.jsx, AuthContext. The request reaches POST /api/auth/login, POST /api/auth/register, GET /api/auth/me. On the server, authController checks user, bcrypt password, email verification, signs JWT, sets cookie. The main data involved is User: password hash, role, verified, activeSessions. The final result is: httpOnly cookie + safe user object + role-based redirect.

Main files: authController.js; middleware/auth.js; User.js; Login.jsx; AuthContext.jsx

Feature 2 - Three-Tier Role System with Composite Priority Queue

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



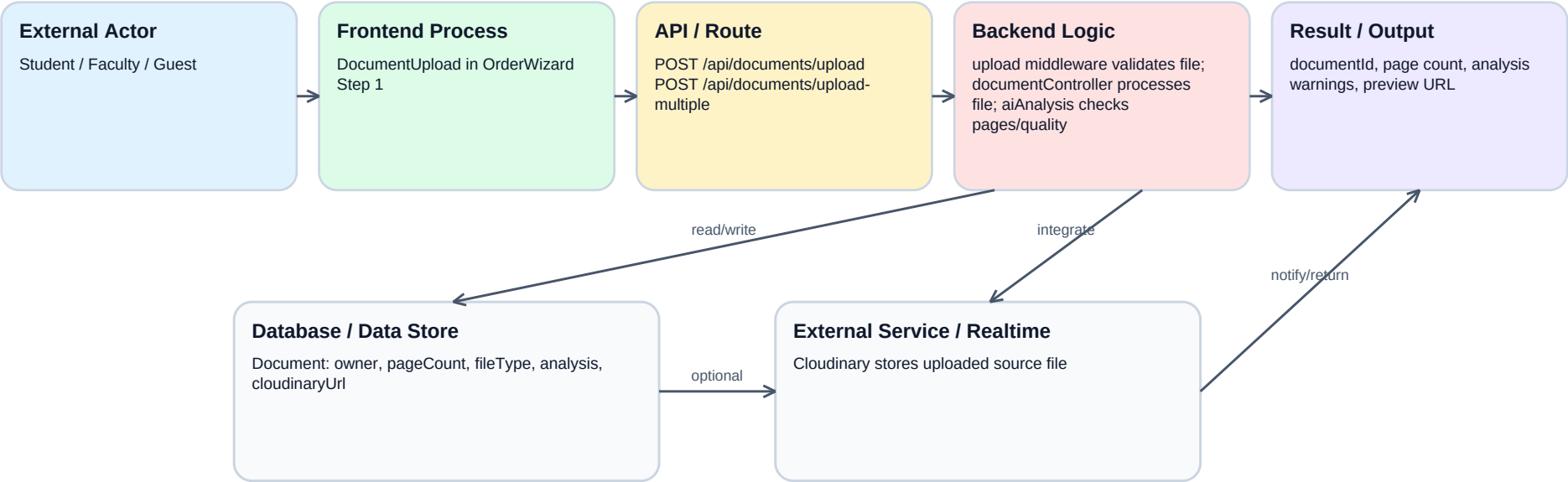
How to explain this feature

I start from Student / Faculty / Staff / Admin. The UI part is ProtectedRoute and App.jsx choose allowed pages. The request reaches Protected routes under /api/admin, /api/staff, /api/orders. On the server, verifyToken attaches req.user; roleGuard checks role; queueService sorts priority. The main data involved is User.role, User.queueAssignment, Order.priorityScore. The final result is: Correct dashboard access and priority-aware queue ordering.

Main files: ProtectedRoute.jsx; roleGuard.js; queueService.js; App.jsx

Feature 3 - Document Upload with AI Analysis

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



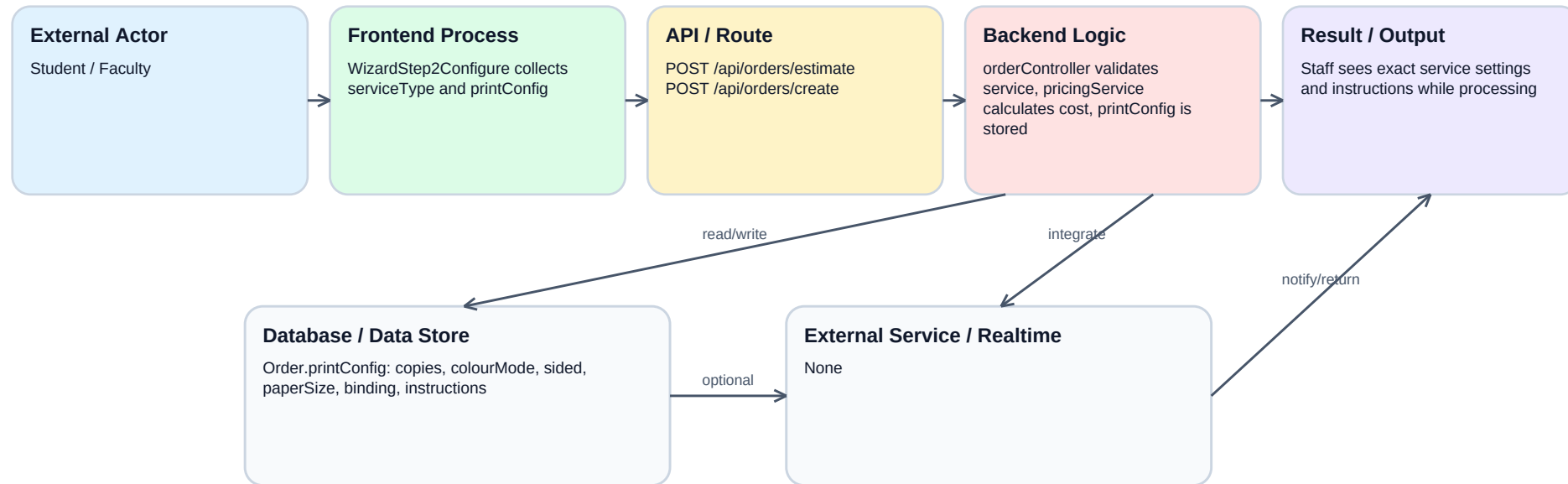
How to explain this feature

I start from Student / Faculty / Guest. The UI part is DocumentUpload in OrderWizard Step 1. The request reaches POST /api/documents/upload, POST /api/documents/upload-multiple. On the server, upload middleware validates file; documentController processes file; aiAnalysis checks pages/quality. The main data involved is Document: owner, pageCount, fileType, analysis, cloudinaryUrl. The final result is: documentId, page count, analysis warnings, preview URL.

Main files: DocumentUpload.jsx; documentController.js; upload.js; aiAnalysis.js; Document.js

Feature 4 - Document Processing Services + Per-Order Print Instructions

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



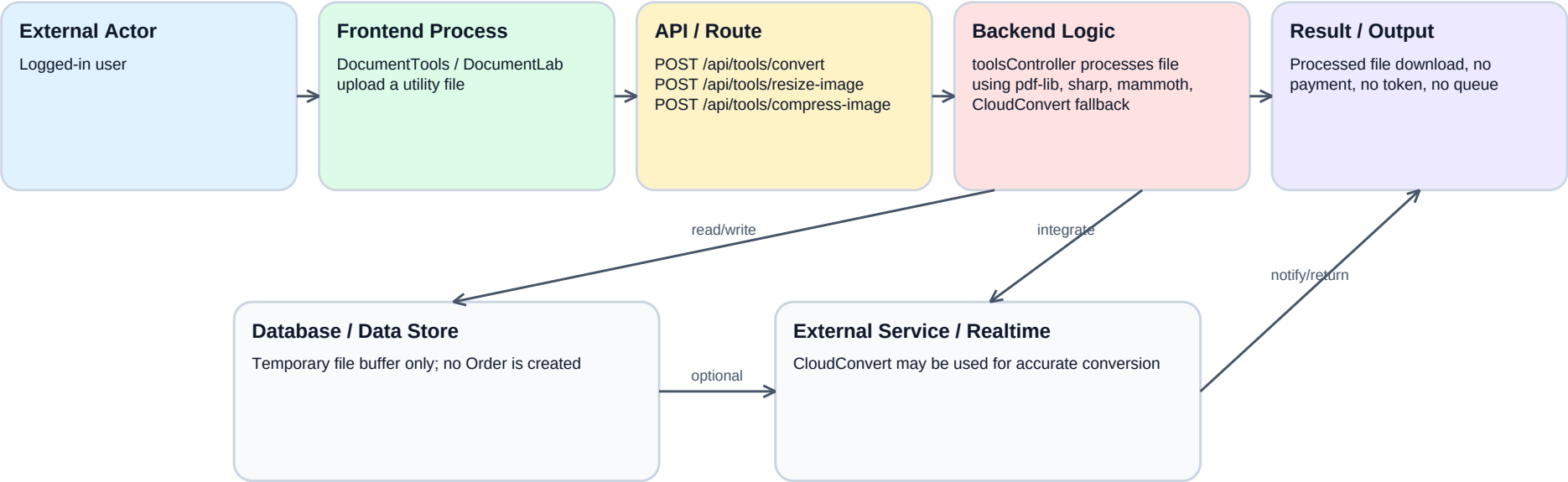
How to explain this feature

I start from Student / Faculty. The UI part is WizardStep2Configure collects serviceType and printConfig. The request reaches POST /api/orders/estimate, POST /api/orders/create. On the server, orderController validates service, pricingService calculates cost, printConfig is stored. The main data involved is Order.printConfig: copies, colourMode, sided, paperSize, binding, instructions. The final result is: Staff sees exact service settings and instructions while processing.

Main files: WizardStep2Configure.jsx; WizardStep3Payment.jsx; Order.js; orderController.js

Feature 5 - Document Tools - Free Utility Suite

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



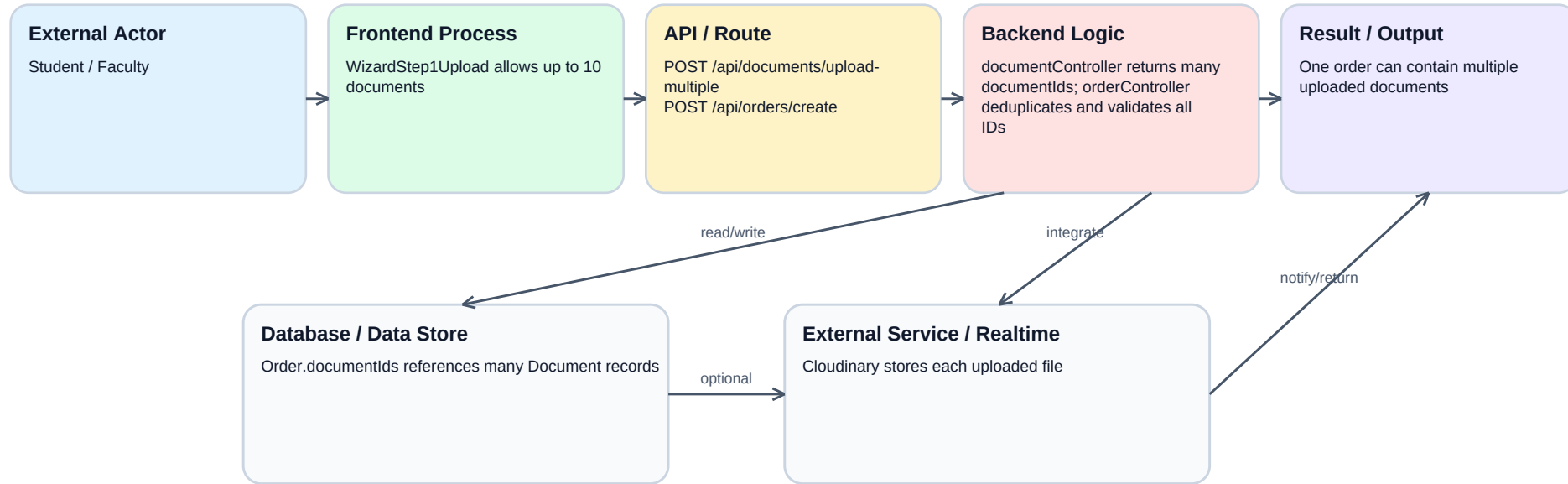
How to explain this feature

I start from Logged-in user. The UI part is DocumentTools / DocumentLab upload a utility file. The request reaches POST /api/tools/convert, POST /api/tools/resize-image, POST /api/tools/compress-image. On the server, toolsController processes file using pdf-lib, sharp, mammoth, CloudConvert fallback. The main data involved is Temporary file buffer only; no Order is created. The final result is: Processed file download, no payment, no token, no queue.

Main files: DocumentTools.jsx; DocumentLab.jsx; routes/tools.js; toolsController.js

Feature 6 - Multi-Document Orders

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



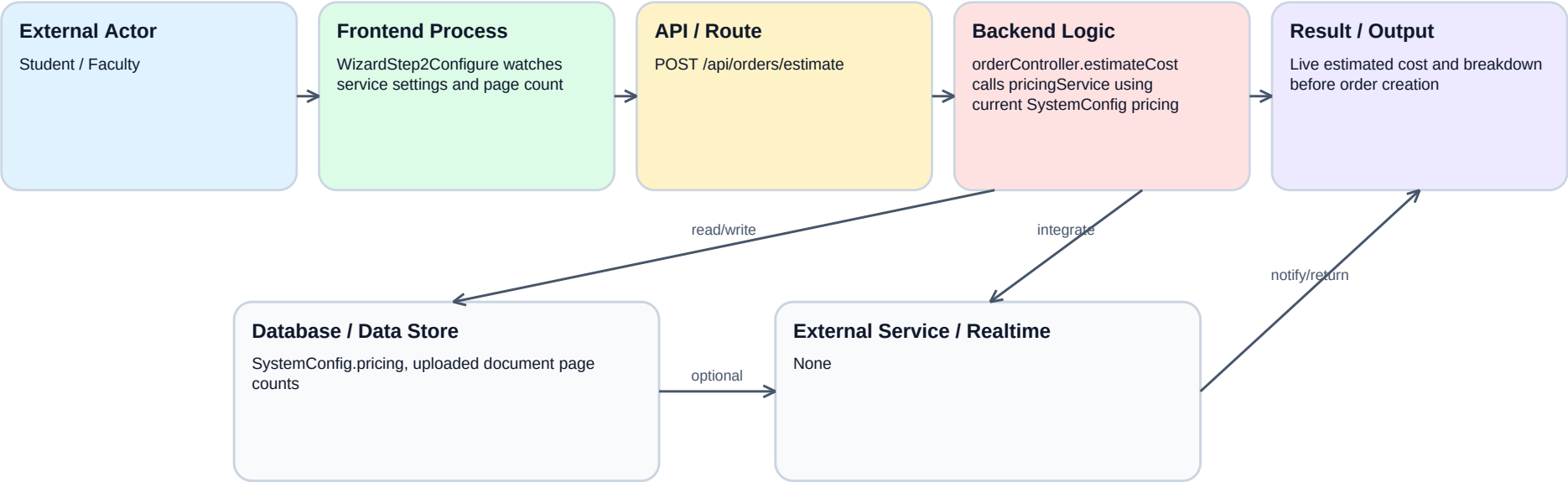
How to explain this feature

I start from Student / Faculty. The UI part is WizardStep1Upload allows up to 10 documents. The request reaches POST /api/documents/upload-multiple, POST /api/orders/create. On the server, documentController returns many documentIds; orderController deduplicates and validates all IDs. The main data involved is Order.documentIds references many Document records. The final result is: One order can contain multiple uploaded documents.

Main files: WizardStep1Upload.jsx; documentController.js; orderController.js; Order.js

Feature 7 - Smart Cost Estimation with Optimisation Suggestions

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



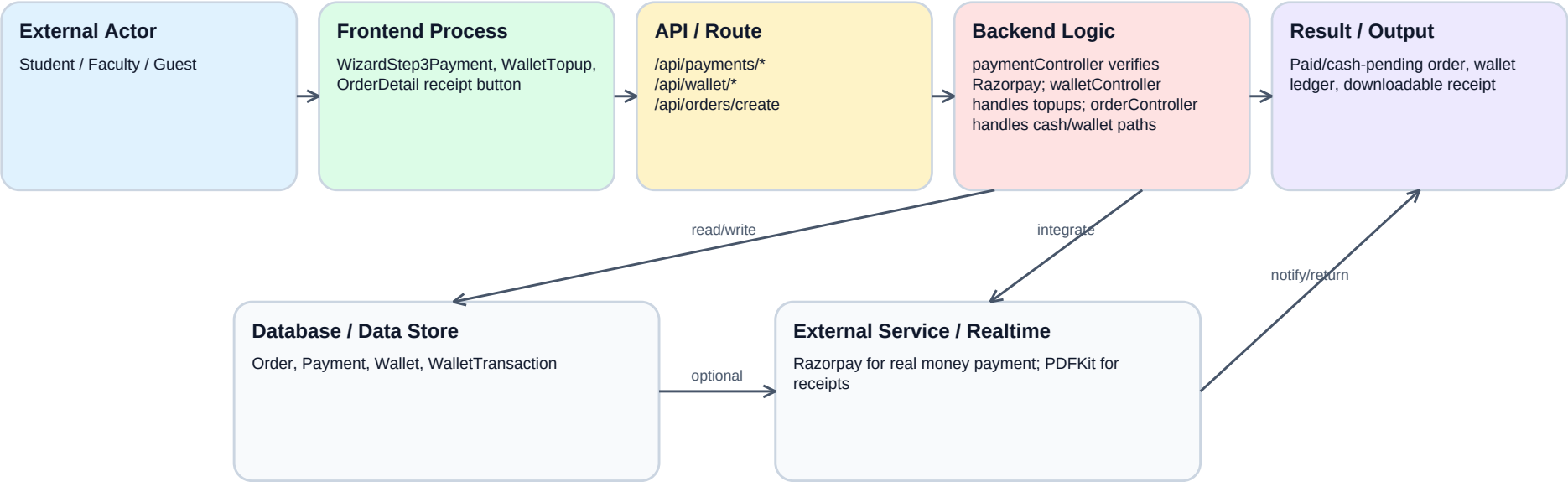
How to explain this feature

I start from Student / Faculty. The UI part is WizardStep2Configure watches service settings and page count. The request reaches POST /api/orders/estimate. On the server, orderController.estimateCost calls pricingService using current SystemConfig pricing. The main data involved is SystemConfig.pricing, uploaded document page counts. The final result is: Live estimated cost and breakdown before order creation.

Main files: WizardStep2Configure.jsx; orderController.js; pricingService.js; SystemConfig.js

Feature 8 - Payment System (Razorpay, Cash, Wallet, Receipts)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



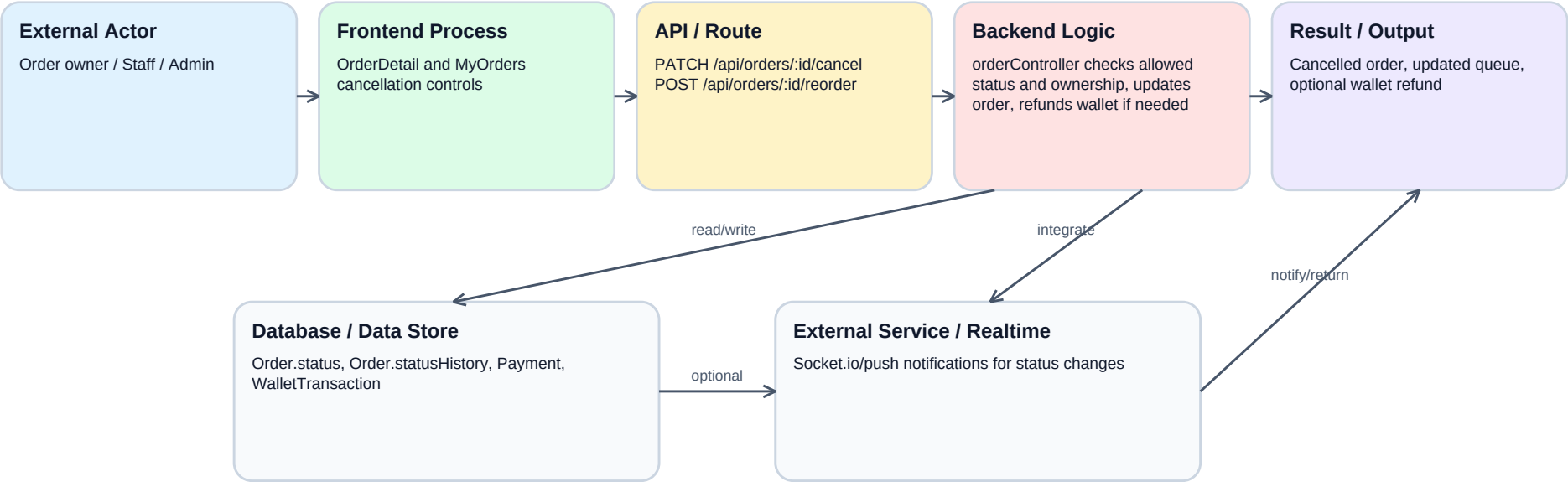
How to explain this feature

I start from Student / Faculty / Guest. The UI part is WizardStep3Payment, WalletTopup, OrderDetail receipt button. The request reaches /api/payments/*, /api/wallet/*, /api/orders/create. On the server, paymentController verifies Razorpay; walletController handles topups; orderController handles cash/wallet paths. The main data involved is Order, Payment, Wallet, WalletTransaction. The final result is: Paid/cash-pending order, wallet ledger, downloadable receipt.

Main files: paymentController.js; walletController.js; orderController.js; Payment.js; WalletTransaction.js

Feature 9 - Order Editing and Cancellation

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



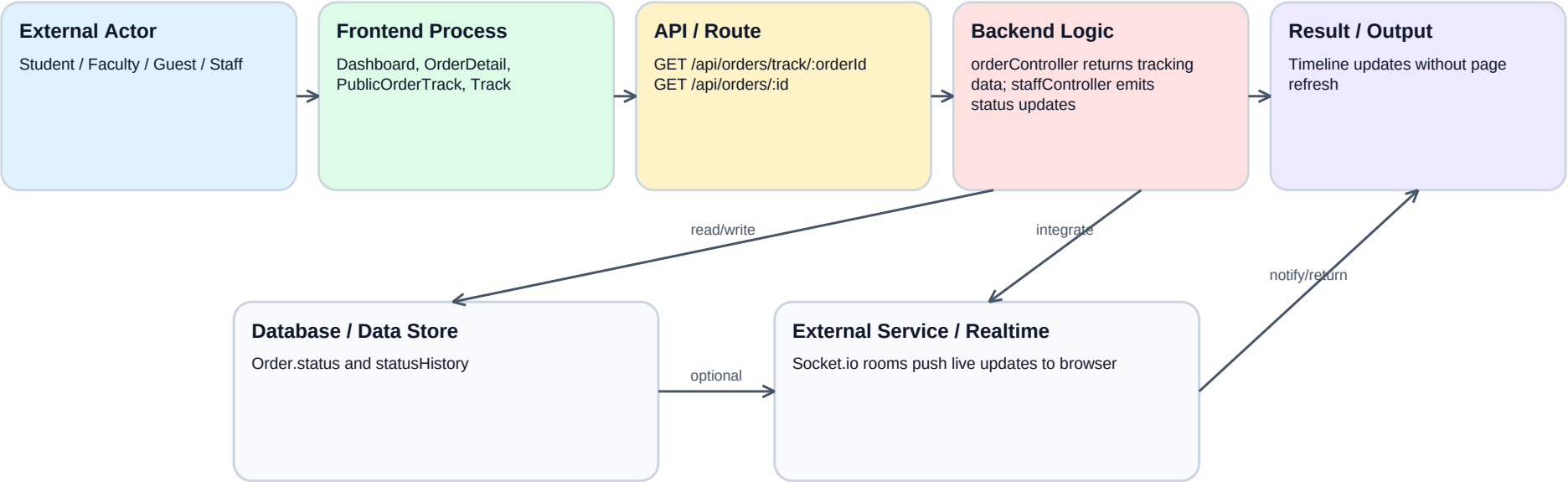
How to explain this feature

I start from Order owner / Staff / Admin. The UI part is OrderDetail and MyOrders cancellation controls. The request reaches PATCH /api/orders/:id/cancel, POST /api/orders/:id/reorder. On the server, orderController checks allowed status and ownership, updates order, refunds wallet if needed. The main data involved is Order.status, Order.statusHistory, Payment, WalletTransaction. The final result is: Cancelled order, updated queue, optional wallet refund.

Main files: OrderDetail.jsx; orderController.js; Order.js; WalletTransaction.js

Feature 10 - Order Tracking with Real-Time Status Timeline

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



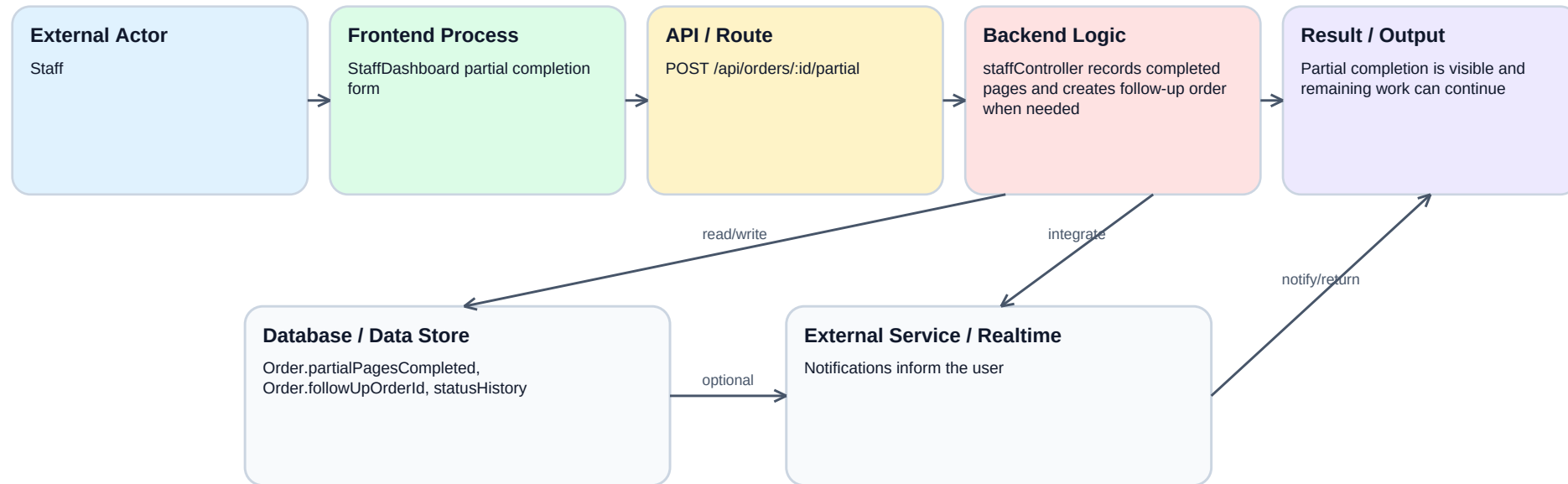
How to explain this feature

I start from Student / Faculty / Guest / Staff. The UI part is Dashboard, OrderDetail, PublicOrderTrack, Track. The request reaches GET /api/orders/track/:orderId, GET /api/orders/:id. On the server, orderController returns tracking data; staffController emits status updates. The main data involved is Order.status and statusHistory. The final result is: Timeline updates without page refresh.

Main files: Dashboard.jsx; PublicOrderTrack.jsx; orderController.js; socketHandler.js

Feature 11 - Partial Order Handling

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



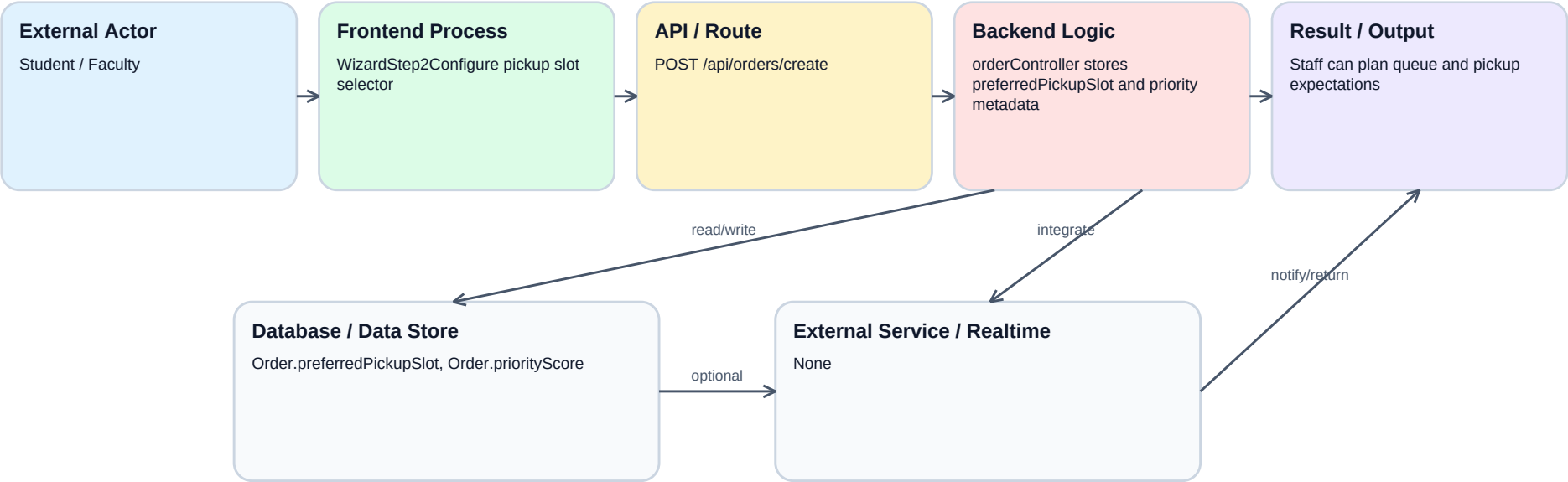
How to explain this feature

I start from Staff. The UI part is StaffDashboard partial completion form. The request reaches POST /api/orders/:id/partial. On the server, staffController records completed pages and creates follow-up order when needed. The main data involved is Order.partialPagesCompleted, Order.followUpOrderId, statusHistory. The final result is: Partial completion is visible and remaining work can continue.

Main files: StaffDashboard.jsx; staffController.js; Order.js; notificationService.js

Feature 12 - Slot Booking and Preferred Pickup Time

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



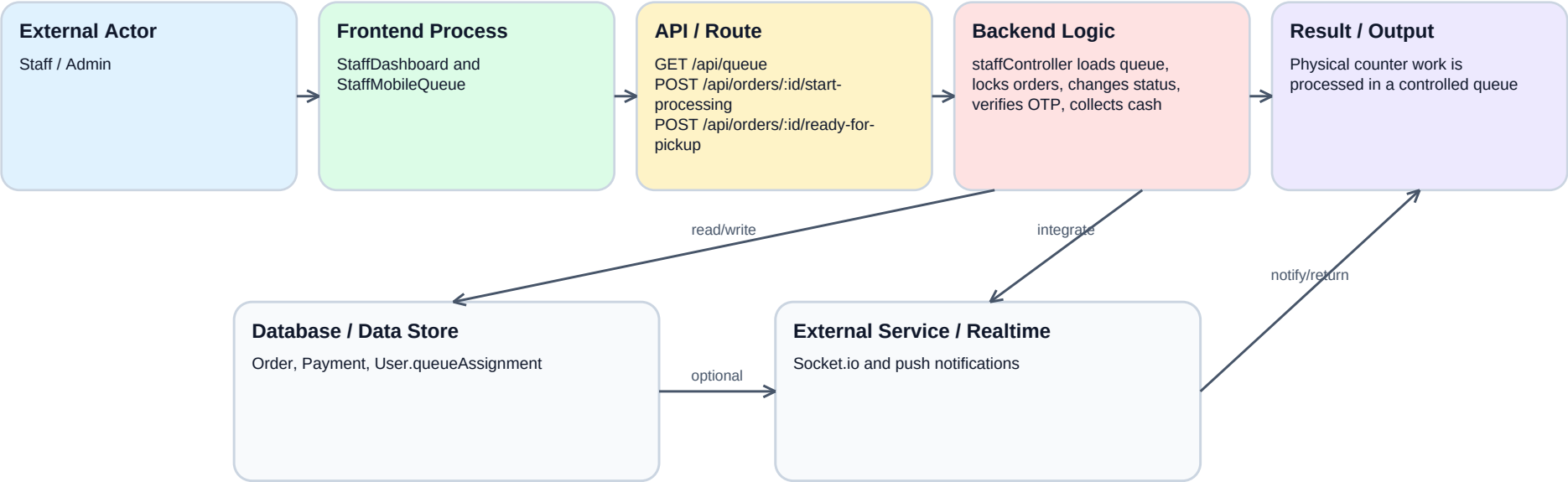
How to explain this feature

I start from Student / Faculty. The UI part is WizardStep2Configure pickup slot selector. The request reaches POST /api/orders/create. On the server, orderController stores preferredPickupSlot and priority metadata. The main data involved is Order.preferredPickupSlot, Order.priorityScore. The final result is: Staff can plan queue and pickup expectations.

Main files: WizardStep2Configure.jsx; orderController.js; Order.js; queueService.js

Feature 13 - Counter Staff Dashboard

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



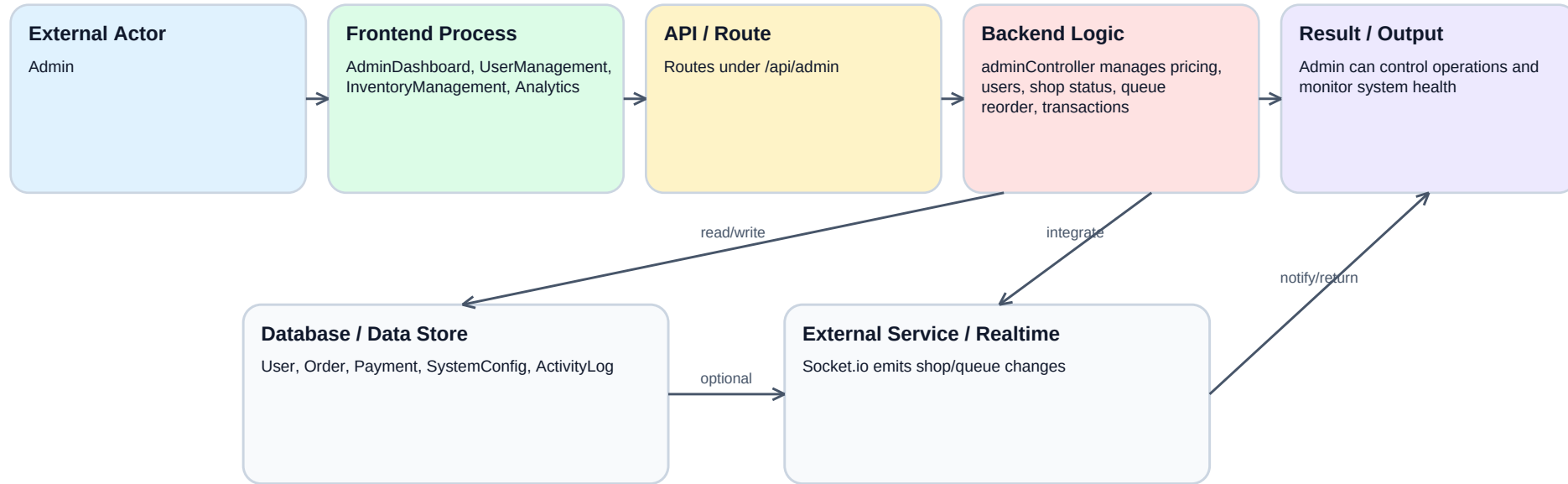
How to explain this feature

I start from Staff / Admin. The UI part is StaffDashboard and StaffMobileQueue. The request reaches GET /api/queue, POST /api/orders/:id/start-processing, POST /api/orders/:id/ready-for-pickup. On the server, staffController loads queue, locks orders, changes status, verifies OTP, collects cash. The main data involved is Order, Payment, User.queueAssignment. The final result is: Physical counter work is processed in a controlled queue.

Main files: StaffDashboard.jsx; routes/staff.js; staffController.js; queueService.js

Feature 14 - Admin Dashboard Management

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



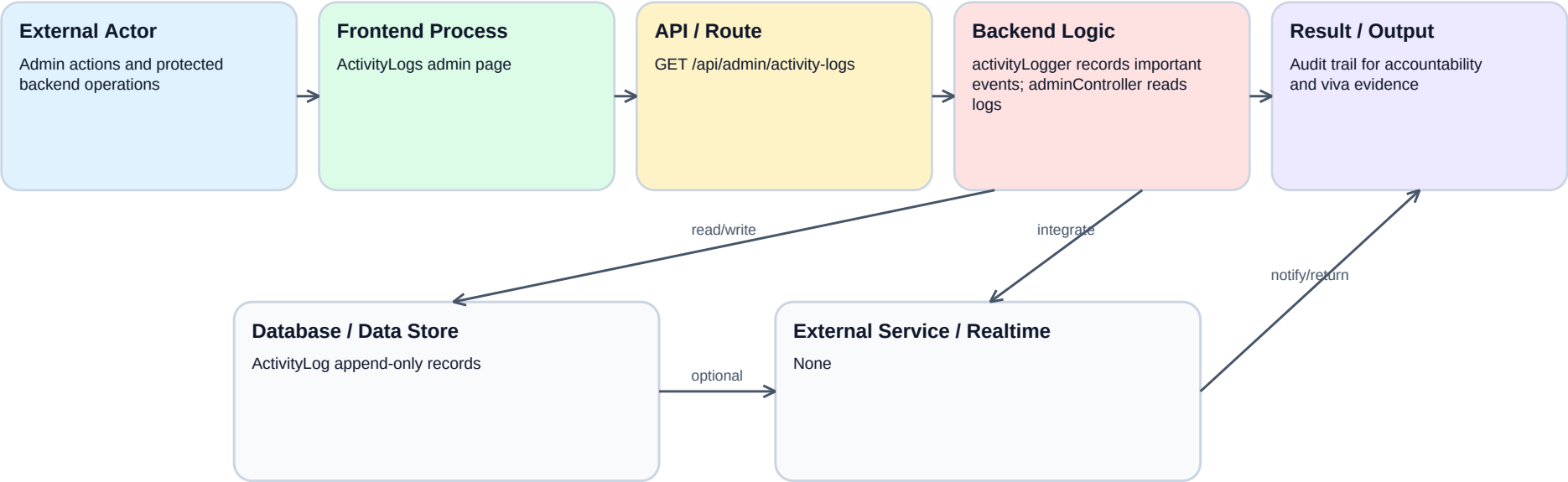
How to explain this feature

I start from Admin. The UI part is AdminDashboard, UserManagement, InventoryManagement, Analytics. The request reaches Routes under /api/admin. On the server, adminController manages pricing, users, shop status, queue reorder, transactions. The main data involved is User, Order, Payment, SystemConfig, ActivityLog. The final result is: Admin can control operations and monitor system health.

Main files: AdminDashboard.jsx; routes/admin.js; adminController.js; SystemConfig.js

Feature 15 - Admin Activity Logs (Append-Only Audit Trail)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



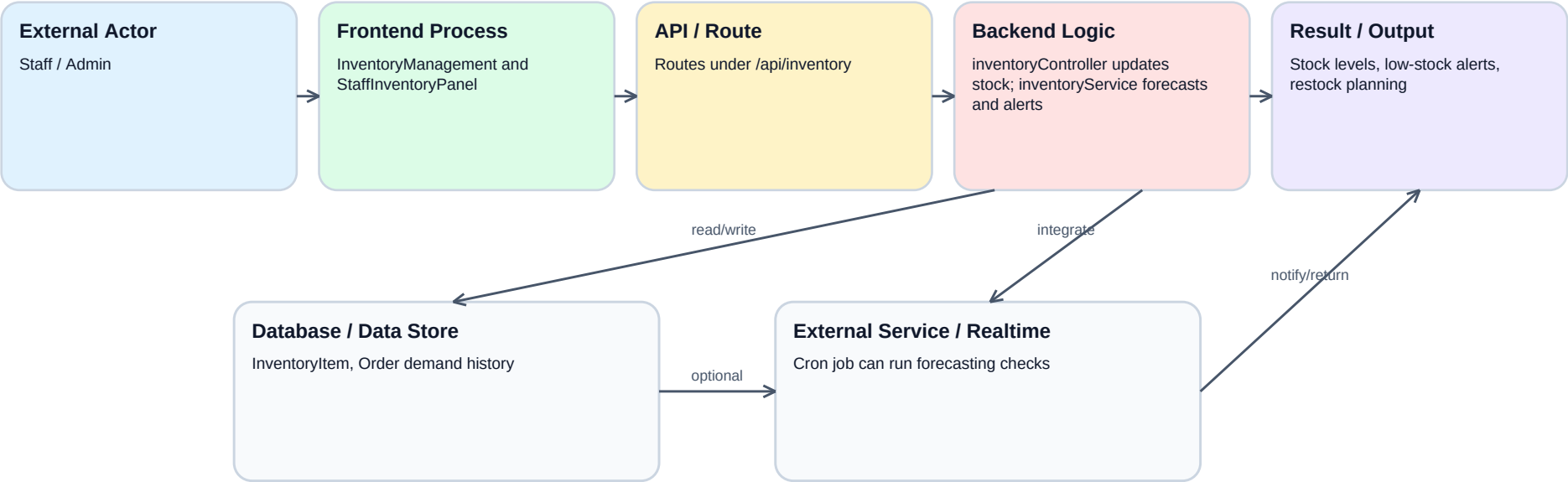
How to explain this feature

I start from Admin actions and protected backend operations. The UI part is ActivityLogs admin page. The request reaches GET /api/admin/activity-logs. On the server, activityLogger records important events; adminController reads logs. The main data involved is ActivityLog append-only records. The final result is: Audit trail for accountability and viva evidence.

Main files: ActivityLogs.jsx; activityLogger.js; ActivityLog.js; adminController.js

Feature 16 - Inventory Tracker with Demand Forecasting

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



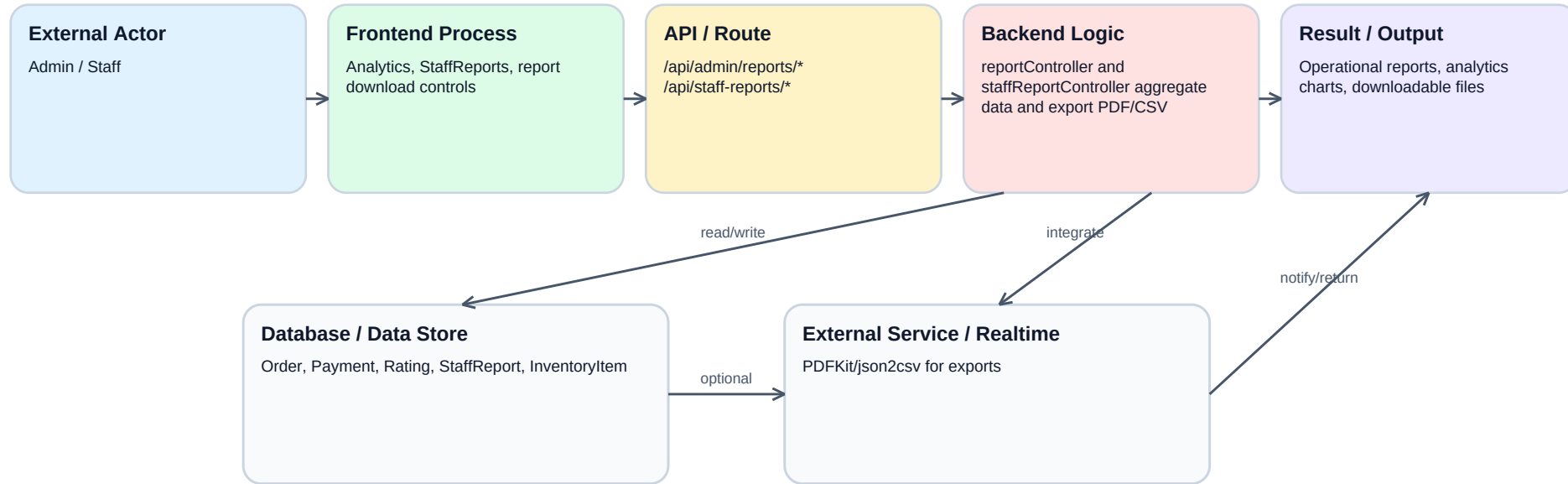
How to explain this feature

I start from Staff / Admin. The UI part is InventoryManagement and StaffInventoryPanel. The request reaches Routes under /api/inventory. On the server, inventoryController updates stock; inventoryService forecasts and alerts. The main data involved is InventoryItem, Order demand history. The final result is: Stock levels, low-stock alerts, restock planning.

Main files: InventoryManagement.jsx; inventory.js; inventoryController.js; inventoryService.js

Feature 17 - Reports and Analytics

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



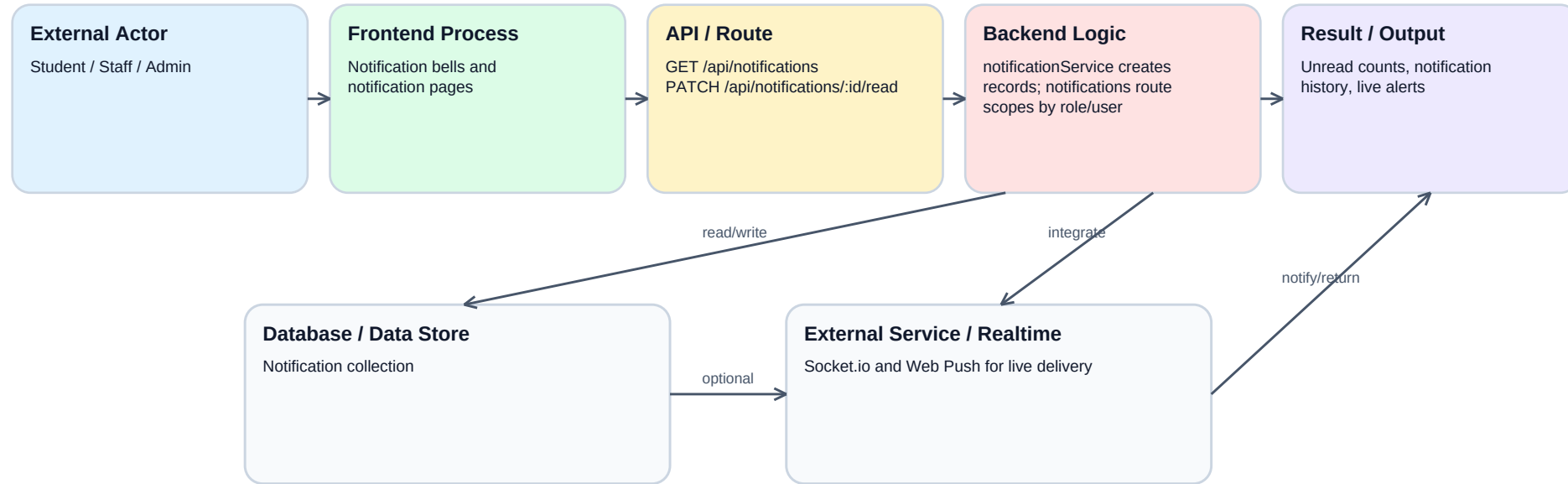
How to explain this feature

I start from Admin / Staff. The UI part is Analytics, StaffReports, report download controls. The request reaches /api/admin/reports/*, /api/staff-reports/*. On the server, reportController and staffReportController aggregate data and export PDF/CSV. The main data involved is Order, Payment, Rating, StaffReport, InventoryItem. The final result is: Operational reports, analytics charts, downloadable files.

Main files: Analytics.jsx; reportController.js; staffReportController.js; StaffReport.js

Feature 18 - Notifications with Persistent History

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



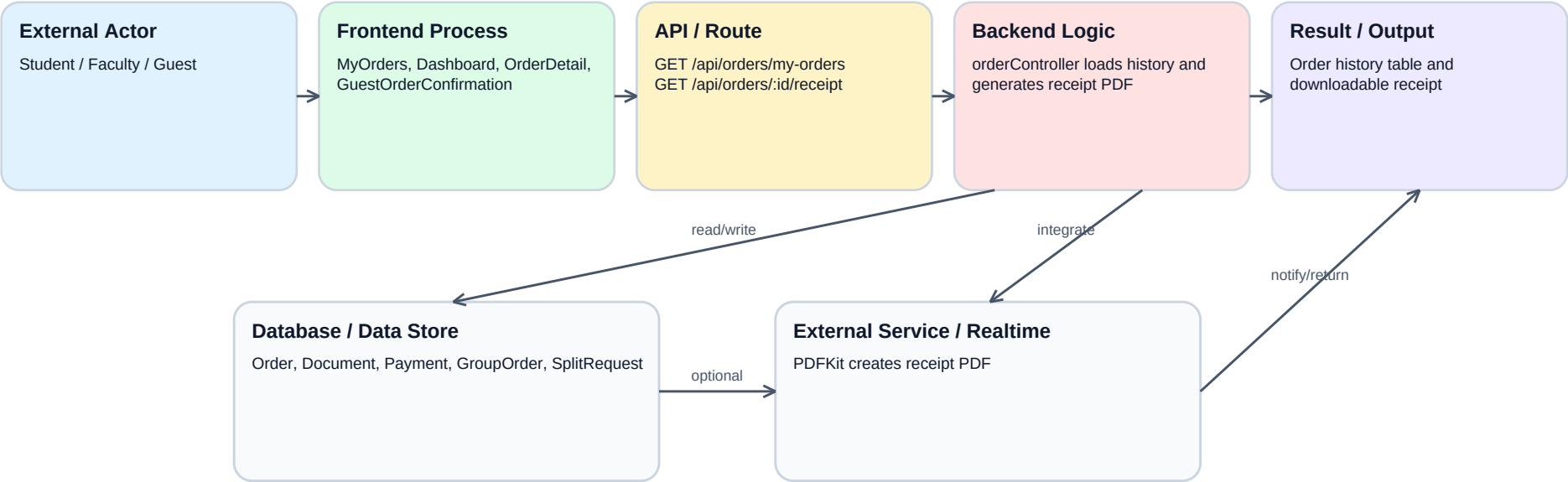
How to explain this feature

I start from Student / Staff / Admin. The UI part is Notification bells and notification pages. The request reaches GET /api/notifications, PATCH /api/notifications/:id/read. On the server, notificationService creates records; notifications route scopes by role/user. The main data involved is Notification collection. The final result is: Unread counts, notification history, live alerts.

Main files: notifications.js; notificationService.js; Notification.js; AdminNotificationBell.jsx

Feature 19 - Order History and Receipt Generation

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



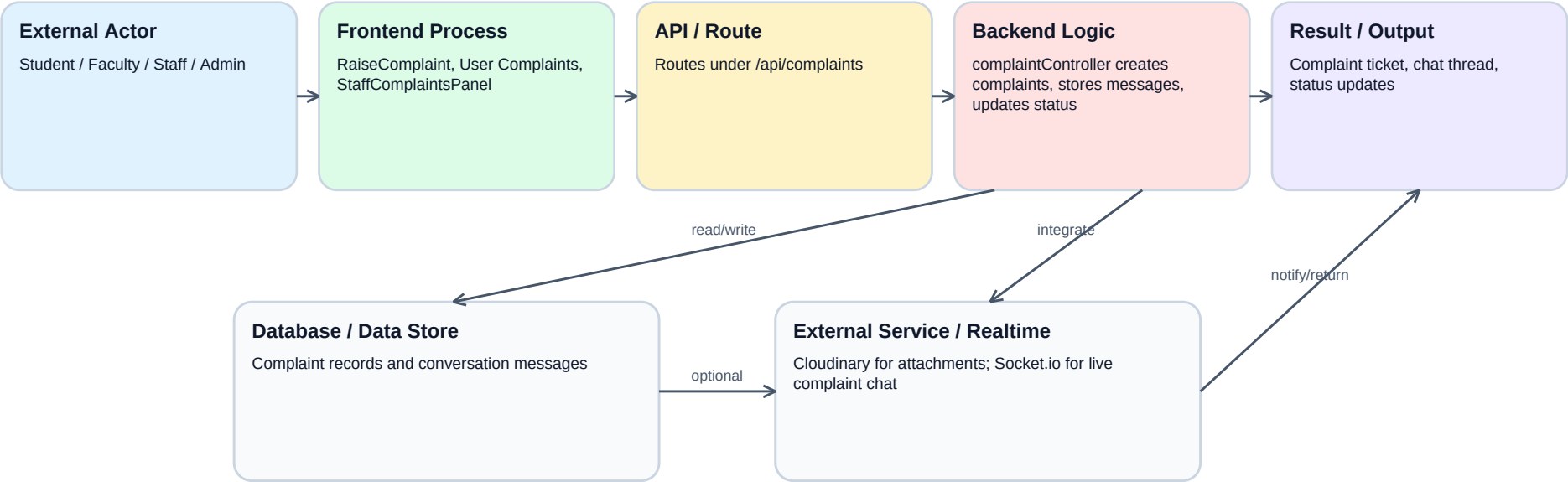
How to explain this feature

I start from Student / Faculty / Guest. The UI part is MyOrders, Dashboard, OrderDetail, GuestOrderConfirmation. The request reaches GET /api/orders/my-orders, GET /api/orders/:id/receipt. On the server, orderController loads history and generates receipt PDF. The main data involved is Order, Document, Payment, GroupOrder, SplitRequest. The final result is: Order history table and downloadable receipt.

Main files: MyOrders.jsx; OrderDetail.jsx; orderController.js; receiptRoutes.js

Feature 20 - Complaint and Messaging System (Real-Time Chat)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



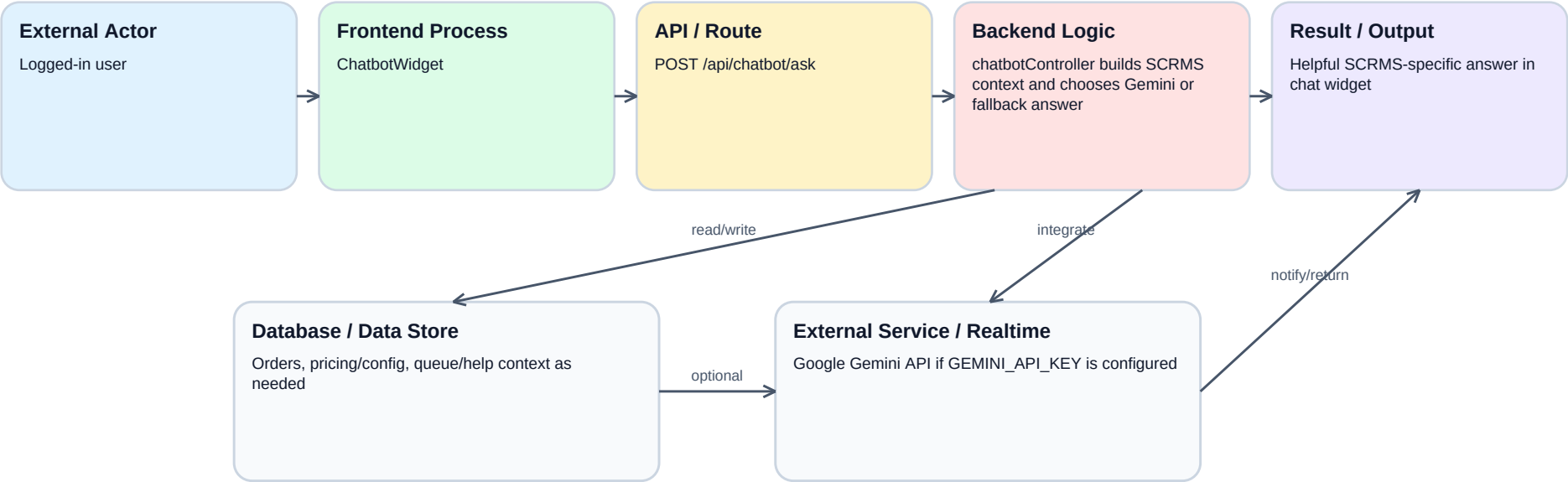
How to explain this feature

I start from Student / Faculty / Staff / Admin. The UI part is RaiseComplaint, User Complaints, StaffComplaintsPanel. The request reaches Routes under /api/complaints. On the server, complaintController creates complaints, stores messages, updates status. The main data involved is Complaint records and conversation messages. The final result is: Complaint ticket, chat thread, status updates.

Main files: complaintController.js; complaints.js; Complaint.js; ComplaintConversationPanel.jsx

Feature 21 - AI Chatbot Assistant (Gemini API + Live DB Context)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



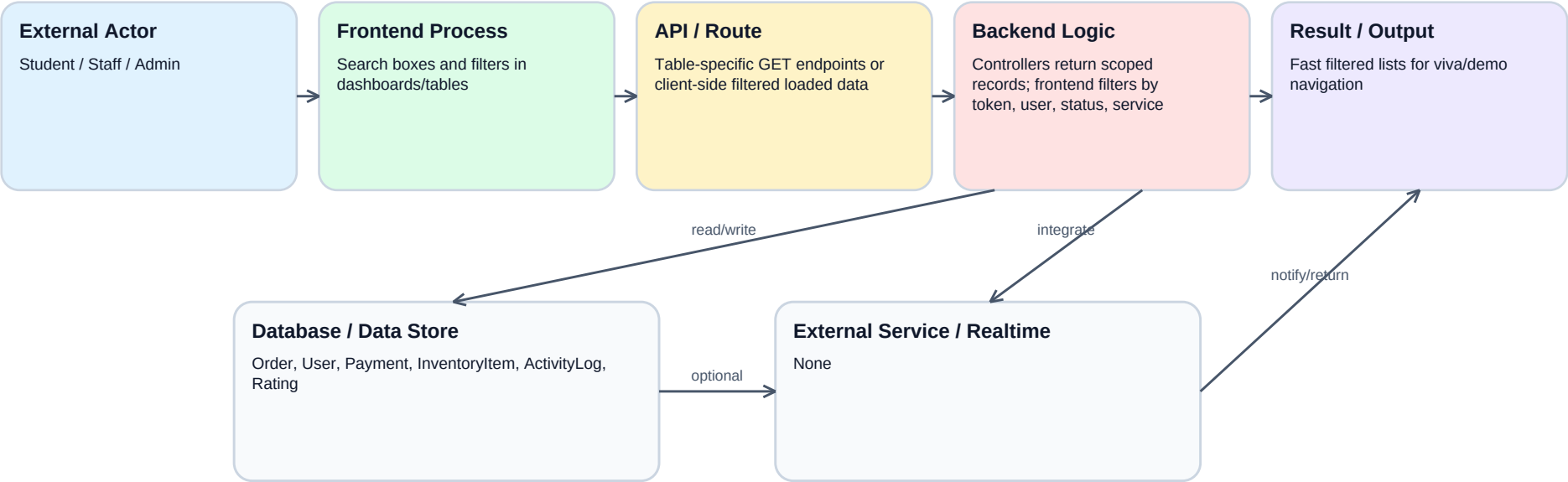
How to explain this feature

I start from Logged-in user. The UI part is ChatbotWidget. The request reaches POST /api/chatbot/ask. On the server, chatbotController builds SCRMS context and chooses Gemini or fallback answer. The main data involved is Orders, pricing/config, queue/help context as needed. The final result is: Helpful SCRMS-specific answer in chat widget.

Main files: ChatbotWidget.jsx; chatbotRoutes.js; chatbotController.js

Feature 22 - Search and Filtering - All System Tables

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



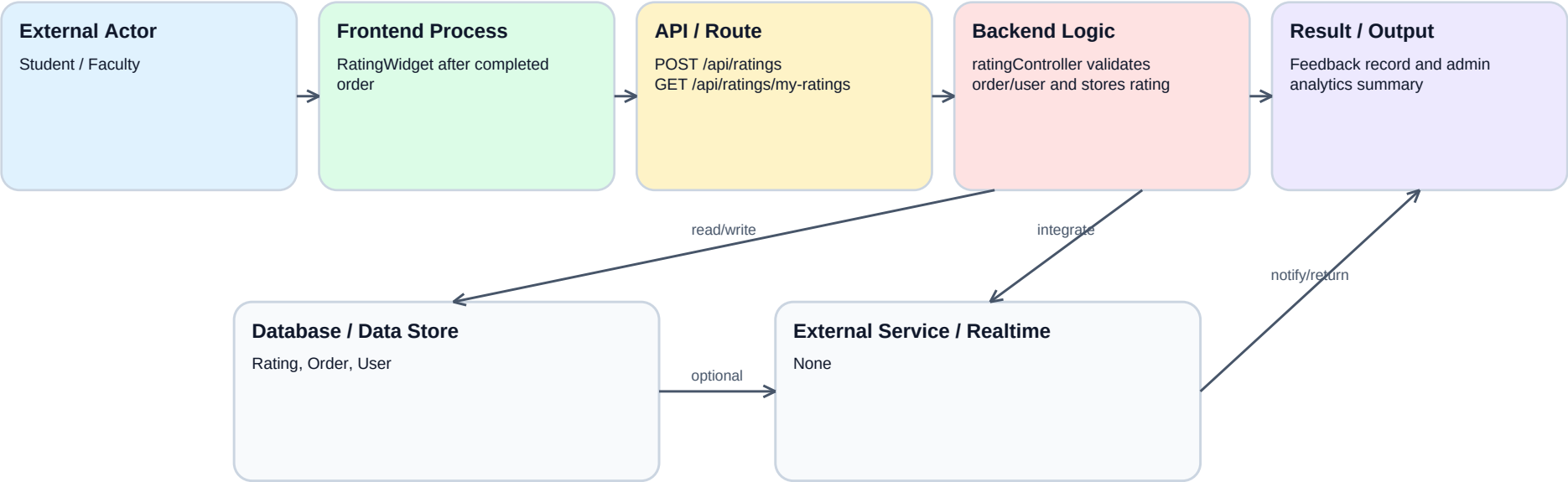
How to explain this feature

I start from Student / Staff / Admin. The UI part is Search boxes and filters in dashboards/tables. The request reaches Table-specific GET endpoints or client-side filtered loaded data. On the server, Controllers return scoped records; frontend filters by token, user, status, service. The main data involved is Order, User, Payment, InventoryItem, ActivityLog, Rating. The final result is: Fast filtered lists for viva/demo navigation.

Main files: Dashboard.jsx; AdminDashboard.jsx; Analytics.jsx; UserManagement.jsx

Feature 23 - User Feedback and Rating System

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



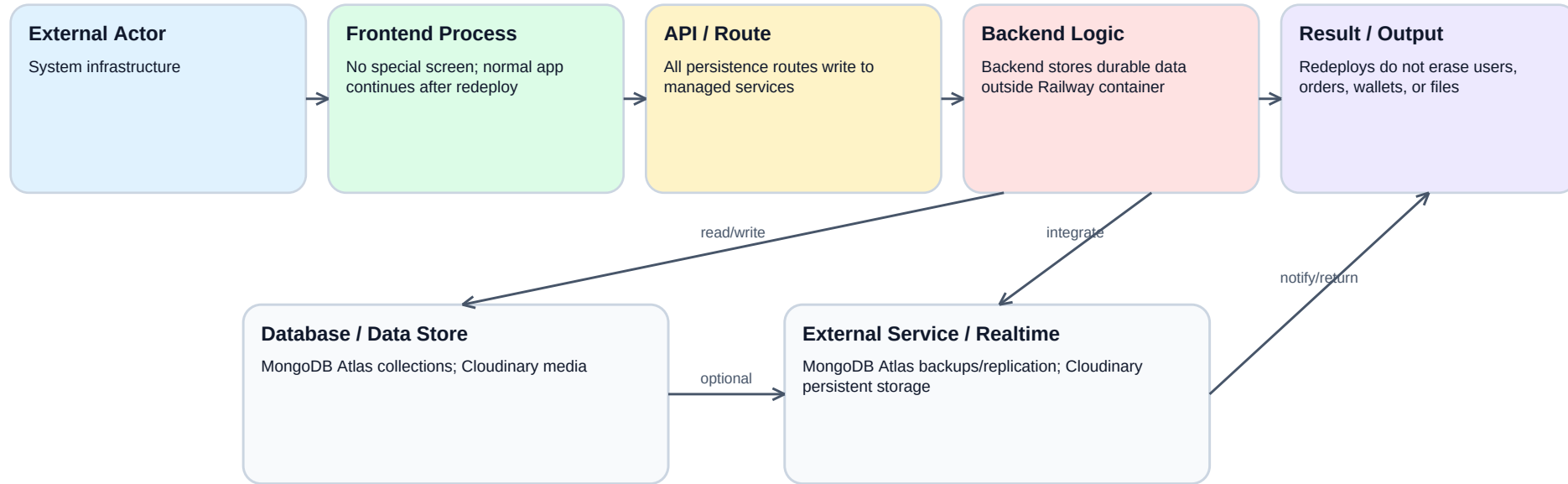
How to explain this feature

I start from Student / Faculty. The UI part is RatingWidget after completed order. The request reaches POST /api/ratings, GET /api/ratings/my-ratings. On the server, ratingController validates order/user and stores rating. The main data involved is Rating, Order, User. The final result is: Feedback record and admin analytics summary.

Main files: RatingWidget.jsx; routes/ratings.js; ratingController.js; Rating.js

Feature 24 - Backup and Recovery Strategy

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



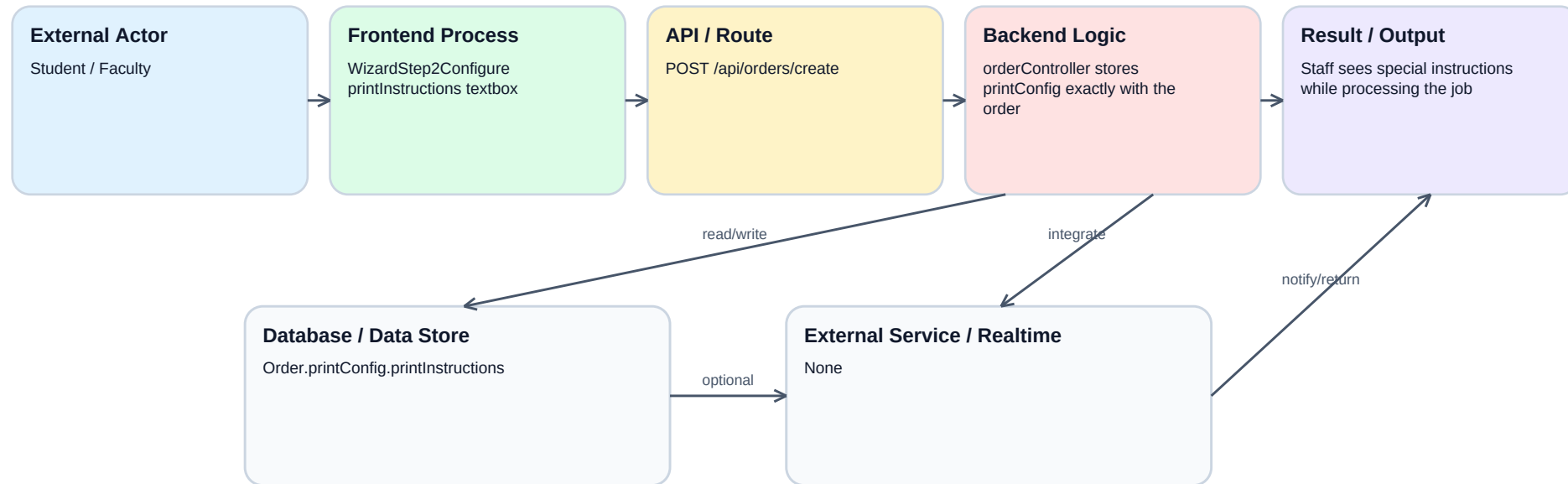
How to explain this feature

I start from System infrastructure. The UI part is No special screen; normal app continues after redeploy. The request reaches All persistence routes write to managed services. On the server, Backend stores durable data outside Railway container. The main data involved is MongoDB Atlas collections; Cloudinary media. The final result is: Redeploys do not erase users, orders, wallets, or files.

Main files: server.js; cloudinaryConfig.js; all Mongoose models

Feature 25 - Print Instructions (Per-Page / Free Text)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



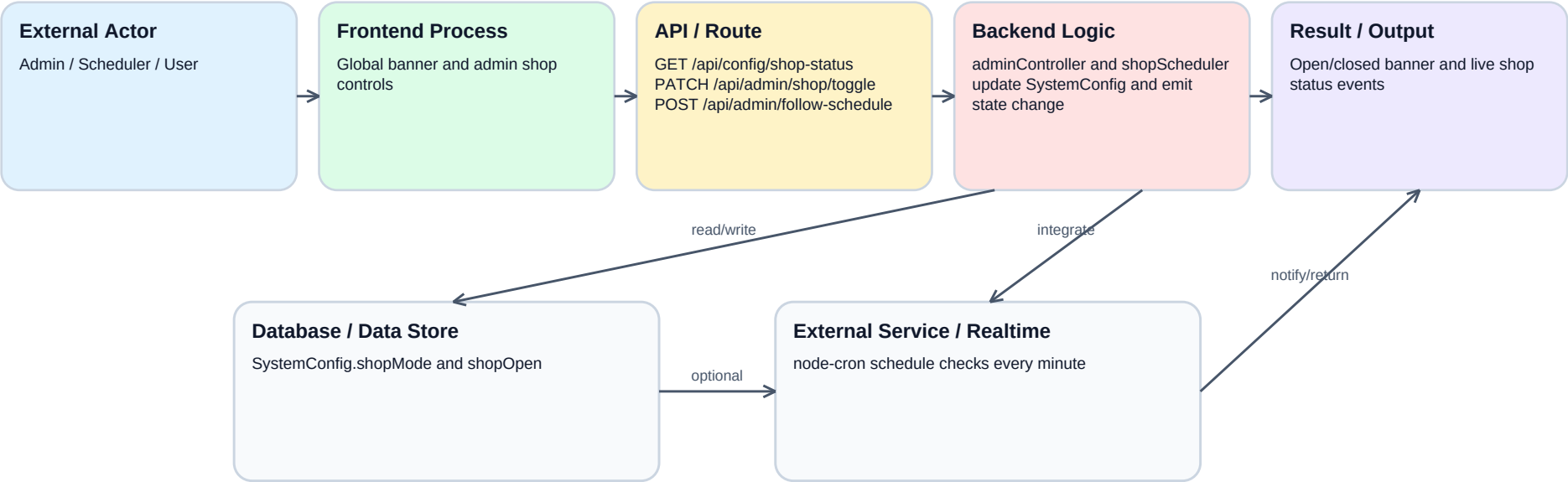
How to explain this feature

I start from Student / Faculty. The UI part is WizardStep2Configure printInstructions textbox. The request reaches POST /api/orders/create. On the server, orderController stores printConfig exactly with the order. The main data involved is Order.printConfig.printInstructions. The final result is: Staff sees special instructions while processing the job.

Main files: WizardStep2Configure.jsx; WizardStep3Payment.jsx; Order.js; orderController.js

Feature 26 - Operating Hours and Shop Open/Closed Status

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



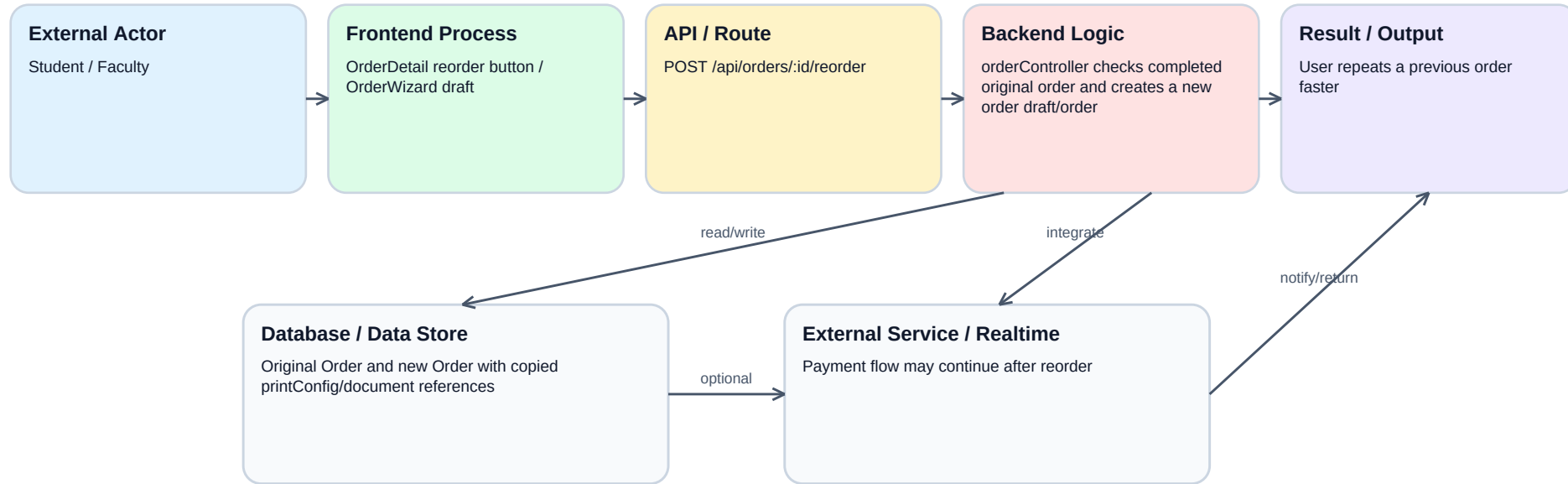
How to explain this feature

I start from Admin / Scheduler / User. The UI part is Global banner and admin shop controls. The request reaches GET /api/config/shop-status, PATCH /api/admin/shop/toggle, POST /api/admin/follow-schedule. On the server, adminController and shopScheduler update SystemConfig and emit state change. The main data involved is SystemConfig.shopMode and shopOpen. The final result is: Open/closed banner and live shop status events.

Main files: shopScheduler.js; adminController.js; configController.js; SystemConfig.js

Feature 27 - Reorder - Repeat a Previous Order

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



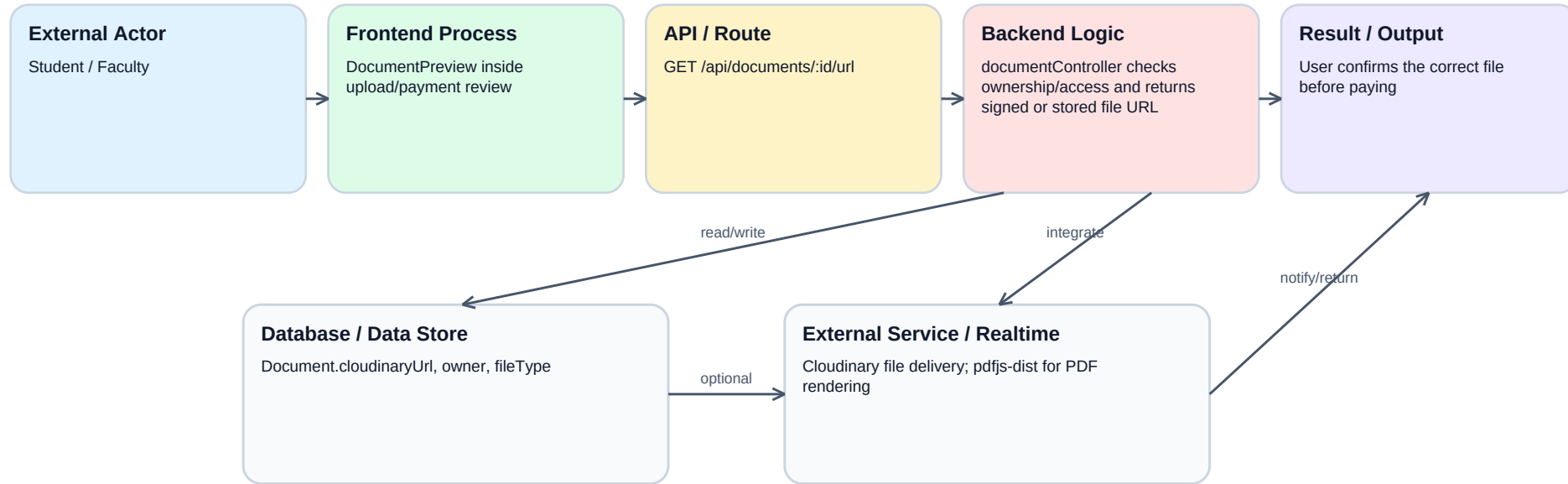
How to explain this feature

I start from Student / Faculty. The UI part is OrderDetail reorder button / OrderWizard draft. The request reaches POST /api/orders/:id/reorder. On the server, orderController checks completed original order and creates a new order draft/order. The main data involved is Original Order and new Order with copied printConfig/document references. The final result is: User repeats a previous order faster.

Main files: OrderDetail.jsx; OrderWizard.jsx; orderController.js; Order.js

Feature 28 - Document Preview Before Payment

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



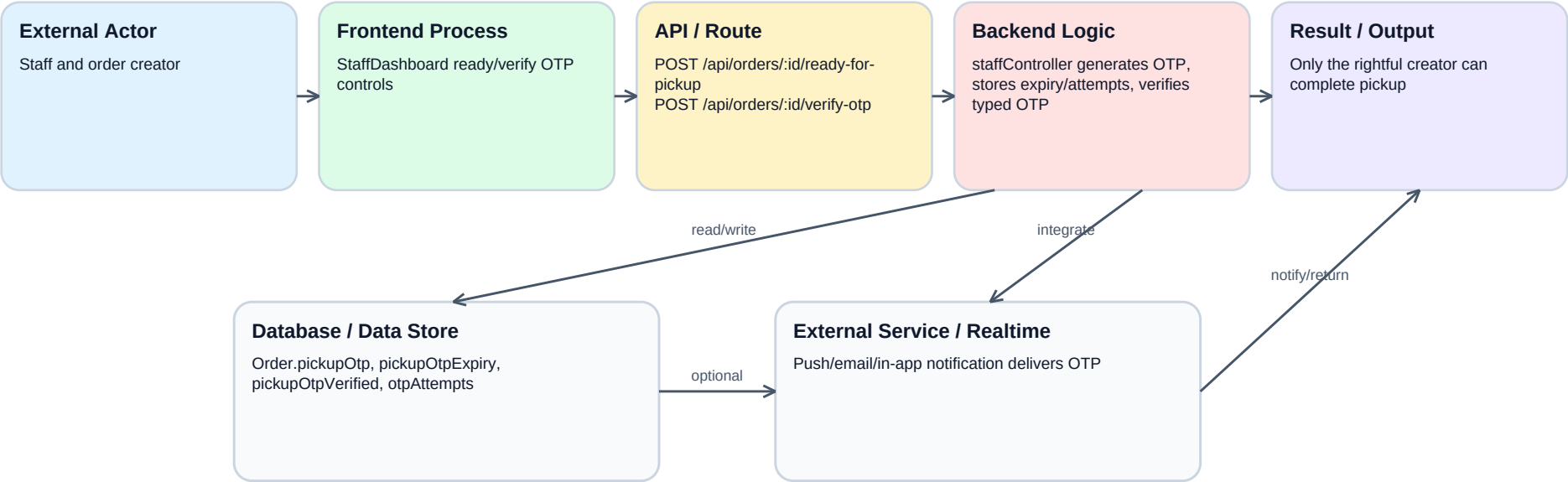
How to explain this feature

I start from Student / Faculty. The UI part is DocumentPreview inside upload/payment review. The request reaches GET /api/documents/:id/url. On the server, documentController checks ownership/access and returns signed or stored file URL. The main data involved is Document.cloudinaryUrl, owner, fileType. The final result is: User confirms the correct file before paying.

Main files: DocumentPreview.jsx; documentAccess.js; documents.js; documentController.js

Feature 29 - Pickup Verification via OTP

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



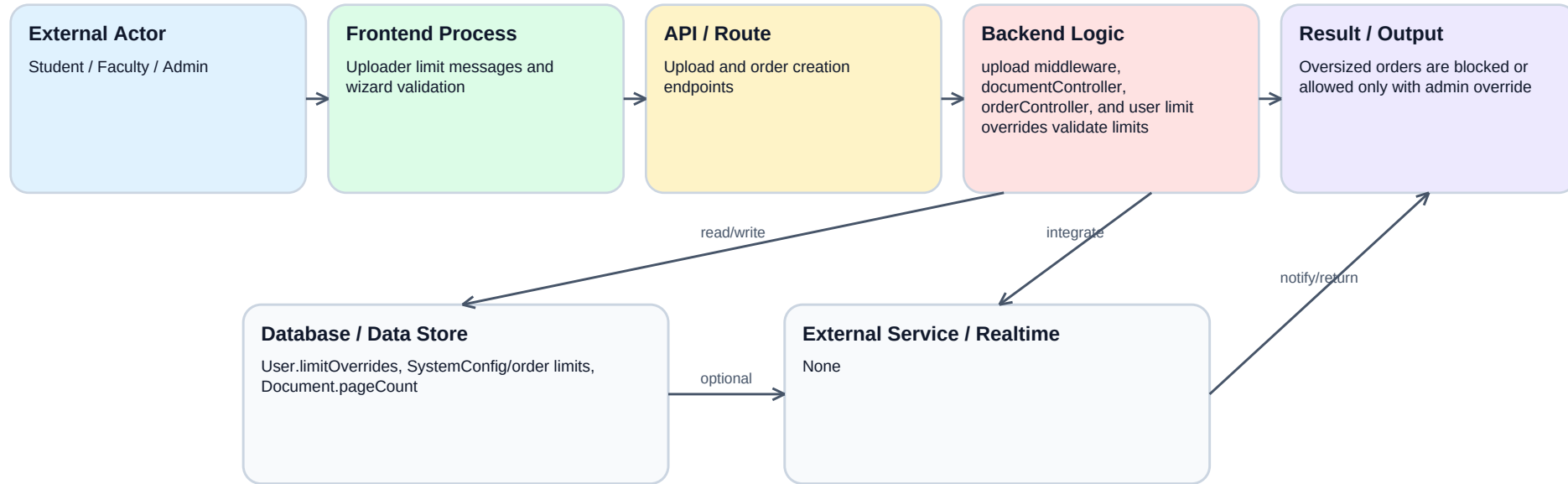
How to explain this feature

I start from Staff and order creator. The UI part is StaffDashboard ready/verify OTP controls. The request reaches POST /api/orders/:id/ready-for-pickup, POST /api/orders/:id/verify-otp. On the server, staffController generates OTP, stores expiry/attempts, verifies typed OTP. The main data involved is Order.pickupOtp, pickupOtpExpiry, pickupOtpVerified, otpAttempts. The final result is: Only the rightful creator can complete pickup.

Main files: StaffDashboard.jsx; staffController.js; Order.js; pushService.js

Feature 30 - Maximum Order Size Limits

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



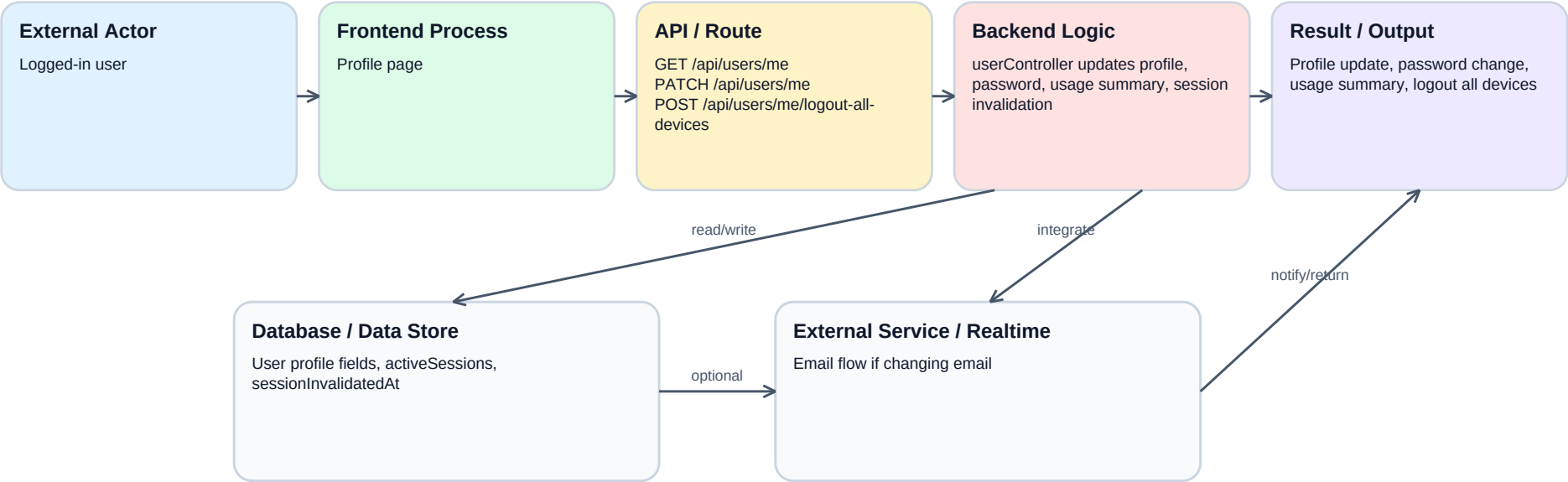
How to explain this feature

I start from Student / Faculty / Admin. The UI part is Uploader limit messages and wizard validation. The request reaches Upload and order creation endpoints. On the server, upload middleware, documentController, orderController, and user limit overrides validate limits. The main data involved is User.limitOverrides, SystemConfig/order limits, Document.pageCount. The final result is: Oversized orders are blocked or allowed only with admin override.

Main files: WizardStep1Upload.jsx; upload.js; orderController.js; User.js

Feature 31 - User Profile Page with Session Management

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



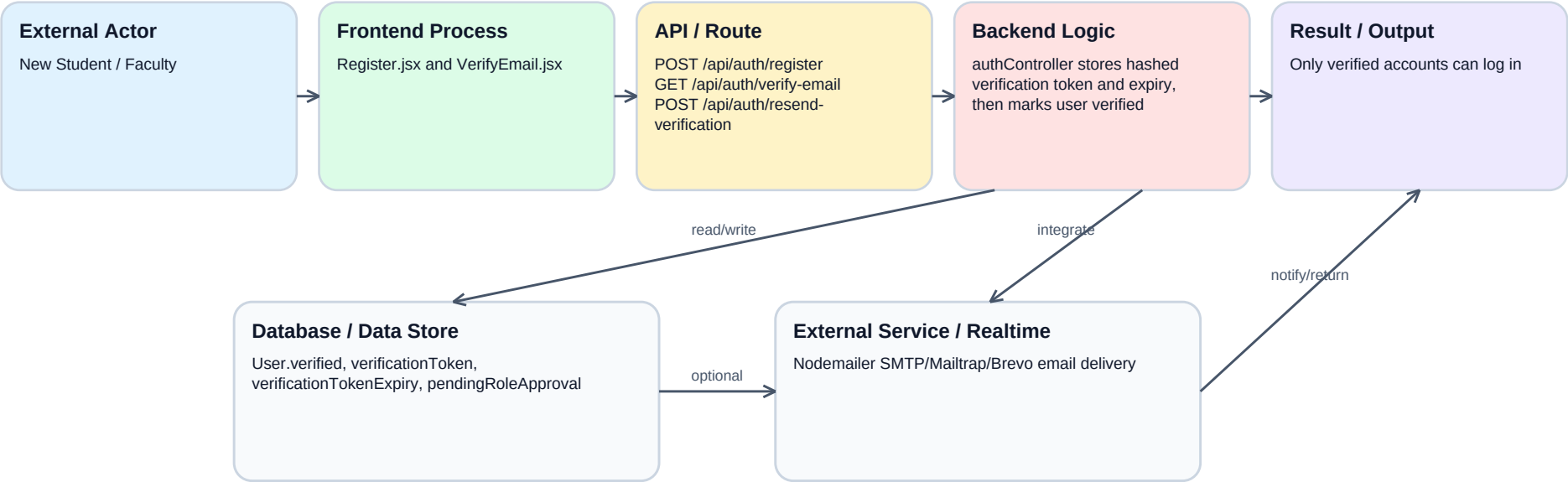
How to explain this feature

I start from Logged-in user. The UI part is Profile page. The request reaches GET /api/users/me, PATCH /api/users/me, POST /api/users/me/logout-all-devices. On the server, userController updates profile, password, usage summary, session invalidation. The main data involved is User profile fields, activeSessions, sessionInvalidatedAt. The final result is: Profile update, password change, usage summary, logout all devices.

Main files: Profile.jsx; routes/users.js; userController.js; User.js

Feature 32 - Email Verification at Registration

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



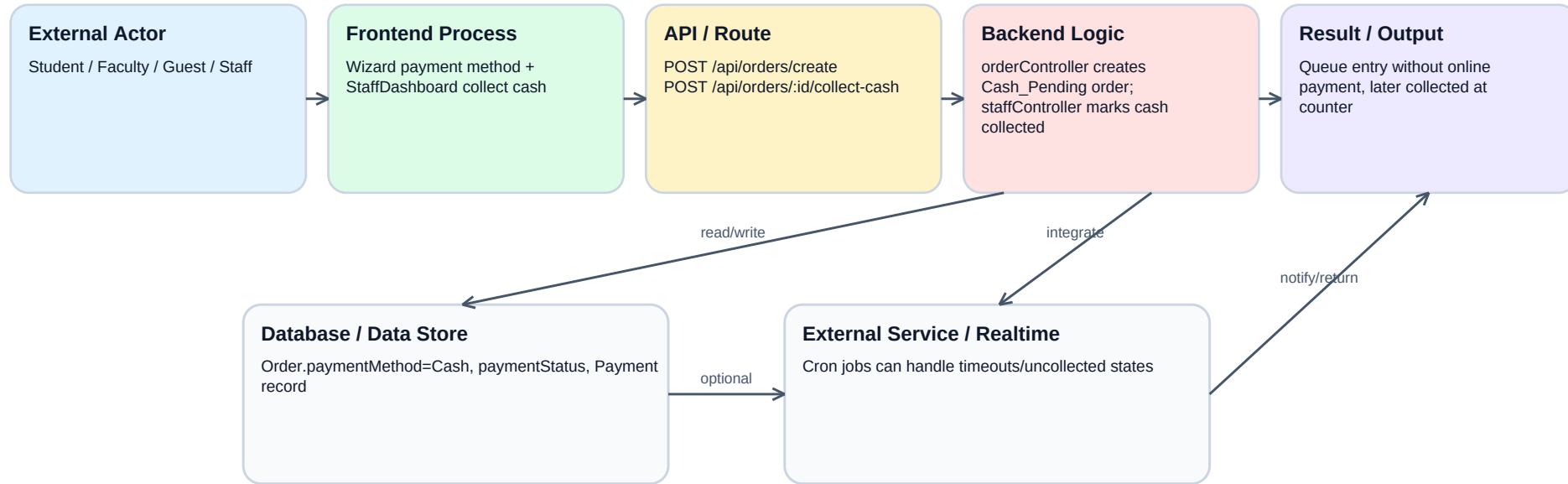
How to explain this feature

I start from New Student / Faculty. The UI part is Register.jsx and VerifyEmail.jsx. The request reaches POST /api/auth/register, GET /api/auth/verify-email, POST /api/auth/resend-verification. On the server, authController stores hashed verification token and expiry, then marks user verified. The main data involved is User.verified, verificationToken, verificationTokenExpiry, pendingRoleApproval. The final result is: Only verified accounts can log in.

Main files: Register.jsx; VerifyEmail.jsx; authController.js; nodemailer.js; User.js

Feature 33 - Pay at Counter - Unpaid, Uncollected and Disputed Handling

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



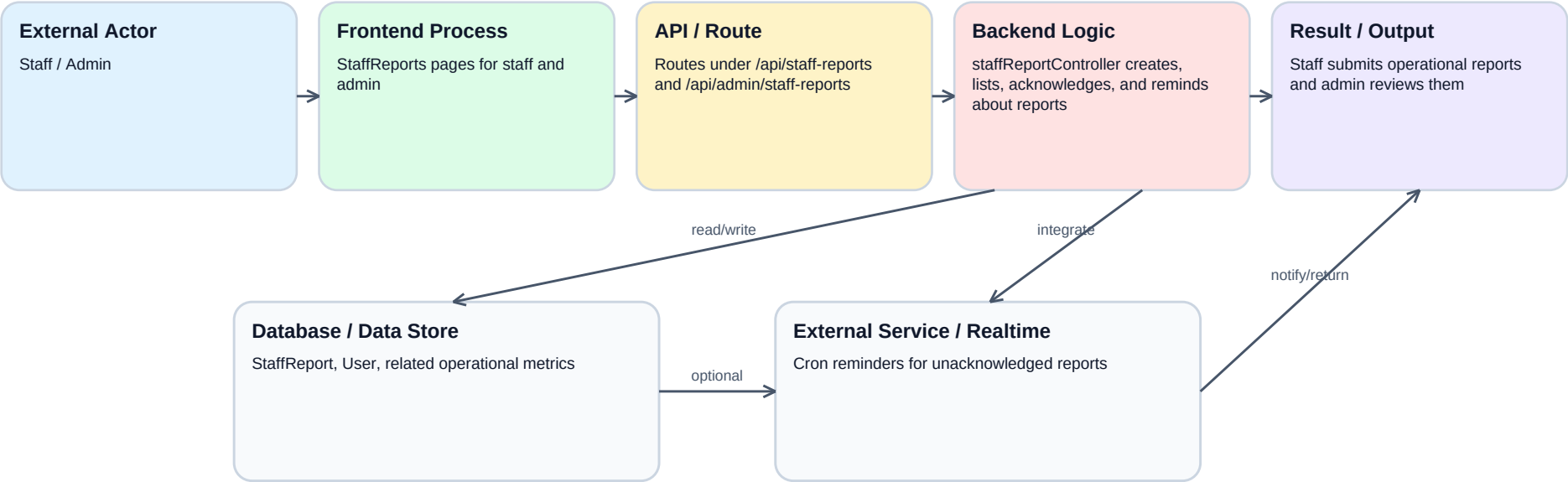
How to explain this feature

I start from Student / Faculty / Guest / Staff. The UI part is Wizard payment method + StaffDashboard collect cash. The request reaches POST /api/orders/create, POST /api/orders/:id/collect-cash. On the server, orderController creates Cash_Pending order; staffController marks cash collected. The main data involved is Order.paymentMethod=Cash, paymentStatus, Payment record. The final result is: Queue entry without online payment, later collected at counter.

Main files: WizardStep3Payment.jsx; orderController.js; staffController.js; Payment.js

Feature 34 - Staff Operational Reporting System

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



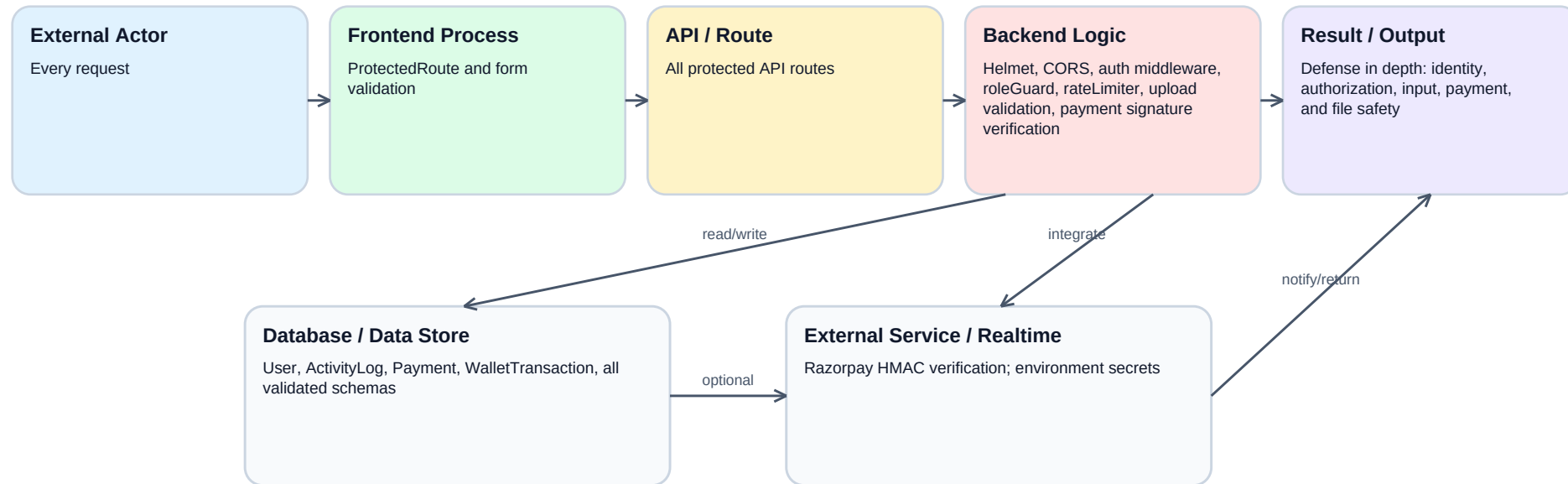
How to explain this feature

I start from Staff / Admin. The UI part is StaffReports pages for staff and admin. The request reaches Routes under /api/staff-reports and /api/admin/staff-reports. On the server, staffReportController creates, lists, acknowledges, and reminds about reports. The main data involved is StaffReport, User, related operational metrics. The final result is: Staff submits operational reports and admin reviews them.

Main files: StaffReports.jsx; routes/staffReports.js; staffReportController.js; StaffReport.js

Feature 35 - Security - Layered Controls

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



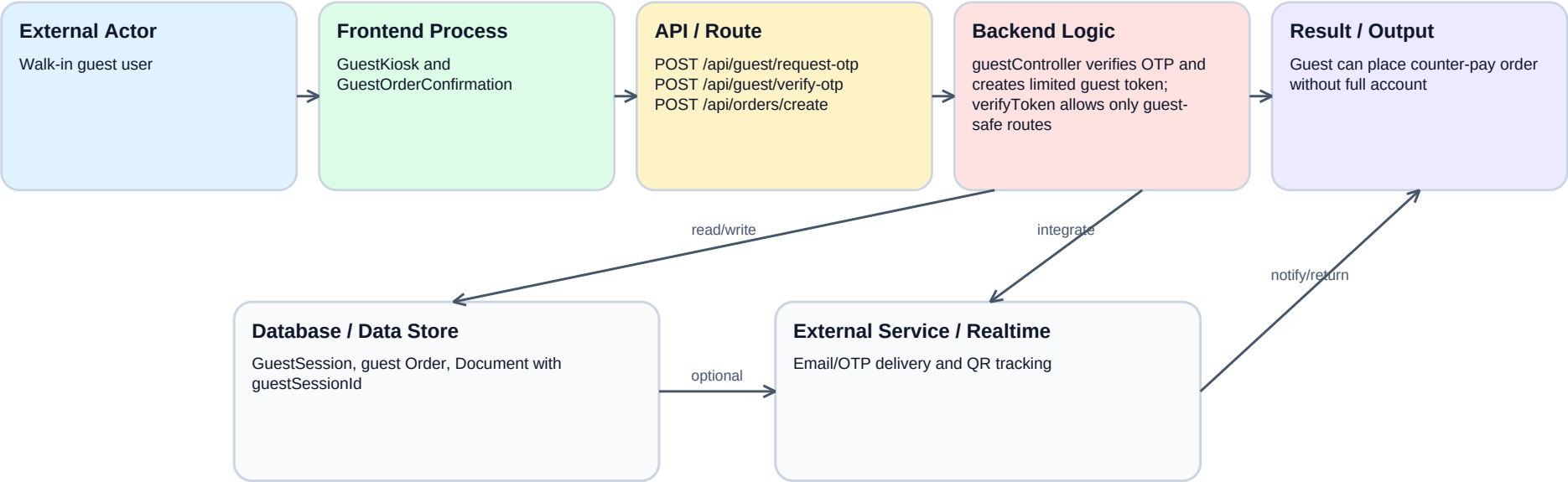
How to explain this feature

I start from Every request. The UI part is ProtectedRoute and form validation. The request reaches All protected API routes. On the server, Helmet, CORS, auth middleware, roleGuard, rateLimiter, upload validation, payment signature verification. The main data involved is User, ActivityLog, Payment, WalletTransaction, all validated schemas. The final result is: Defense in depth: identity, authorization, input, payment, and file safety.

Main files: app.js; auth.js; roleGuard.js; rateLimiter.js; upload.js; paymentController.js

Feature 36 - Decentralised Smart Kiosk Mode (Guest Session)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



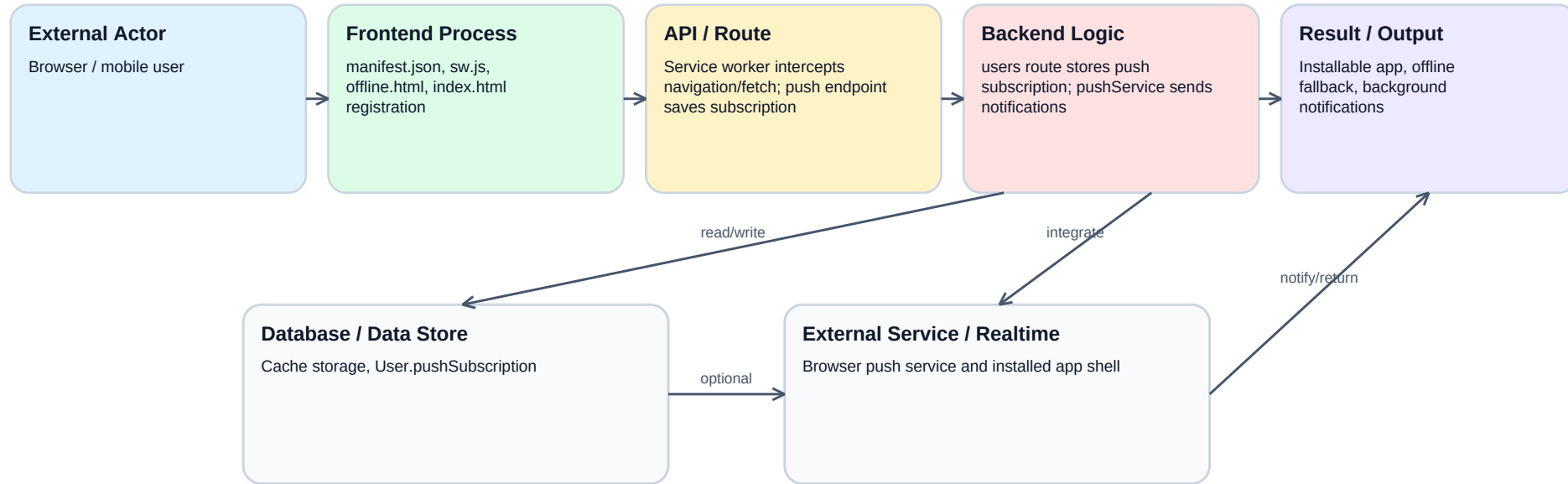
How to explain this feature

I start from Walk-in guest user. The UI part is GuestKiosk and GuestOrderConfirmation. The request reaches POST /api/guest/request-otp, POST /api/guest/verify-otp, POST /api/orders/create. On the server, guestController verifies OTP and creates limited guest token; verifyToken allows only guest-safe routes. The main data involved is GuestSession, guest Order, Document with guestSessionId. The final result is: Guest can place counter-pay order without full account.

Main files: GuestKiosk.jsx; guestController.js; routes/guest.js; GuestSession.js; auth.js

Feature 37 - PWA Architecture (Added Missing Feature)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



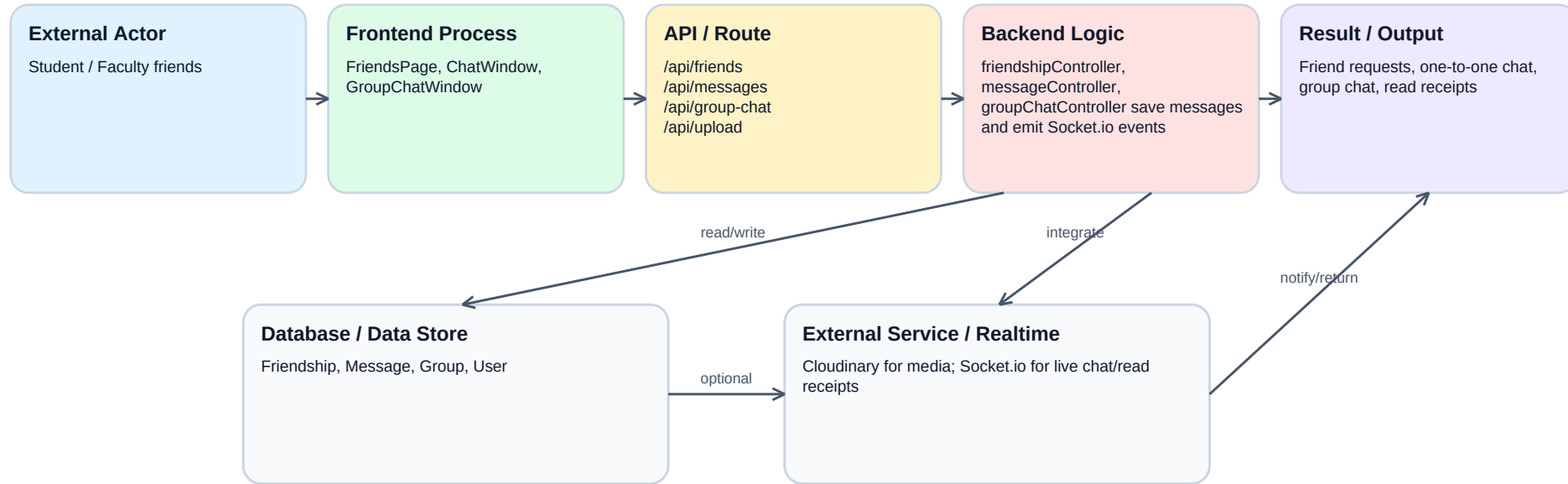
How to explain this feature

I start from Browser / mobile user. The UI part is manifest.json, sw.js, offline.html, index.html registration. The request reaches Service worker intercepts navigation/fetch; push endpoint saves subscription. On the server, users route stores push subscription; pushService sends notifications. The main data involved is Cache storage, User.pushSubscription. The final result is: Installable app, offline fallback, background notifications.

Main files: manifest.json; sw.js; offline.html; usePush.js; pushService.js

Feature 38 - Friend Chat System (Added Missing Feature)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



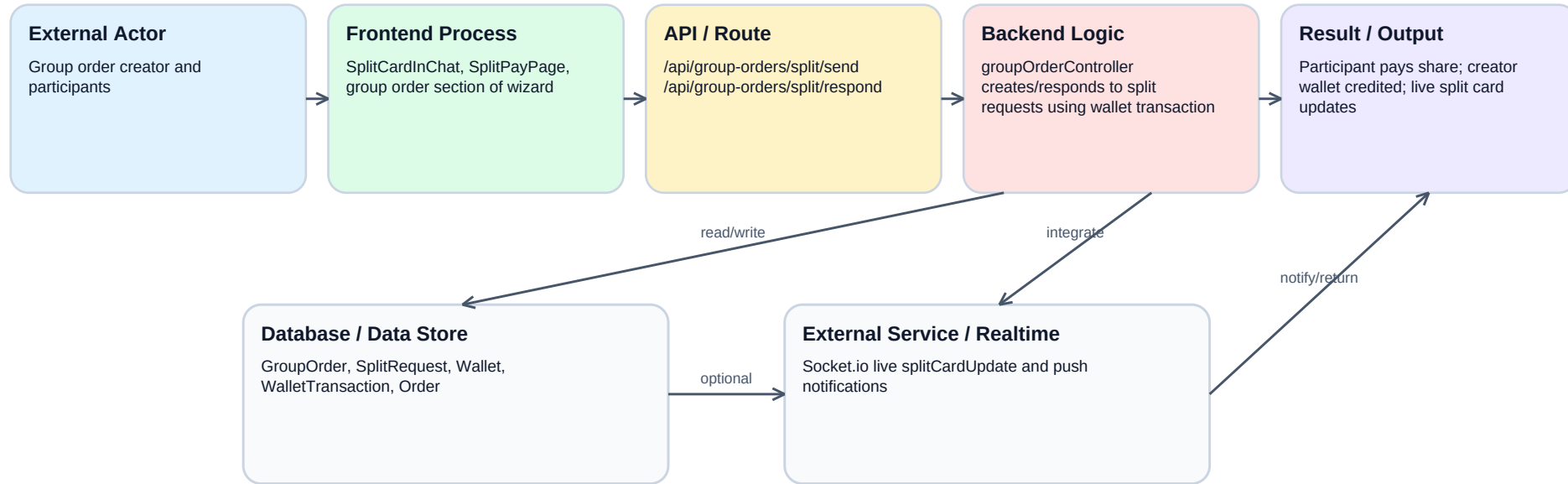
How to explain this feature

I start from Student / Faculty friends. The UI part is FriendsPage, ChatWindow, GroupChatWindow. The request reaches /api/friends, /api/messages, /api/group-chat, /api/upload. On the server, friendshipController, messageController, groupChatController save messages and emit Socket.io events. The main data involved is Friendship, Message, Group, User. The final result is: Friend requests, one-to-one chat, group chat, read receipts.

Main files: Friendship.js; Message.js; Group.js; ChatWindow.jsx; messageController.js

Feature 39 - Split Requests and Live Split Cards (Added Missing Feature)

Data Flow Diagram: actor/input -> frontend -> API/backend -> database/external service -> output



How to explain this feature

I start from Group order creator and participants. The UI part is SplitCardInChat, SplitPayPage, group order section of wizard. The request reaches /api/group-orders/split/send, /api/group-orders/split/respond. On the server, groupOrderController creates/responds to split requests using wallet transaction. The main data involved is GroupOrder, SplitRequest, Wallet, WalletTransaction, Order. The final result is: Participant pays share; creator wallet credited; live split card updates.

Main files: GroupOrder.js; SplitRequest.js; groupOrderController.js; SplitCardInChat.jsx